

Master Thesis

Maschinelles Lernen für Datenimputation, Echtzeit-Rekonstruktion und Synthetische Datengenerierung in der V2X-Kommunikation

Machine Learning for Data Imputation, Real-time Reconstruction and Synthetic Data Generation in V2X Communication

Author: Aditya Jawahar Kanasara

Matriculation Number: 422019

Submission Date: June 5, 2026

Radio Communications and Navigation Department (WICON)

Reviewer: M.Sc. Sanket Partani

Supervisor: Prof. Dr.-Ing. Hans D. Schotten

Declaration

I hereby declare solemnly that I have prepared this thesis according to the BPO/MPO guidelines for Electrical Engineering and Information Technology, except for the help from my advisor. I have fully and correctly cited all sources and resources used, including the Internet. I have clearly indicated all that has been taken from the work of others verbatim, in abbreviated form or in essence.

Kaiserslautern, June 5, 2026

Aditya Jawahar Kansara

Abstract

A major focus in current automotive technology is the communication network that supports automated driving and how vehicles interact with their surroundings. Vehicular communication allows vehicles, infrastructure, and networks to share information. This helps to improve road safety, enhances traffic flow, and supports better decision-making. While vehicular communication is highly data-intensive, it generates large amounts of measurement data under dynamic real-world conditions. Due to changing network conditions, connectivity issues, hardware limitations, and measurement failures, real-world measurement campaigns often contain missing values, data degradation, and data loss.

While recreating such measurement campaigns is neither practical nor cost-effective, there is a need to develop methods that address missing data or generate realistic data to compensate for data loss. Therefore, this thesis begins with a brief overview of Vehicle-to-Everything (V2X) communication systems, V2X access technologies, dataset selection, and machine learning applications in vehicular communication. The cellular dataset from the Berlin V2X measurement campaign is selected as the basis for this work. Building on this foundation, the thesis presents data analysis, model implementation, and evaluation methods to address the identified data challenges through missing-data reconstruction and synthetic-data generation.

First, a GRU-based imputation approach is used to recover missing values of the communication dataset. The completed dataset is then used as input to a GAN-based approach to generate realistic synthetic data to fill missing timestamp intervals encountered during the measurement campaign. The performance metrics and visual comparisons show that the imputation of missing values and the generation of realistic synthetic data were successful. Finally, the limitations of the proposed techniques are discussed, and possible developments are presented for the generation of realistic real-time vehicle communication data.

Contents

Declaration	II
Abstract	III
List of Figures	VI
List of Tables	VII
Abbreviations	VIII
1 Introduction	1
1.1 Problem Statement	2
1.2 Overview of Thesis	3
2 Literature Research	5
2.1 Cooperative Intelligent Transportation Systems	7
2.2 V2X Access Technologies	9
2.2.1 Dedicated Short Range Communication	9
2.2.2 Cellular V2X Communication	10
2.3 Need for V2X Dataset	11
2.4 Machine Learning for V2X Data	12
2.4.1 Application of Machine Learning in V2X	13
2.4.2 Machine Learning Methods for V2X Data	14
2.5 Data Imputation and Data Generation using ML	15
2.5.1 Data Imputation	15
2.5.2 Data Generation	17
2.6 Available V2X Dataset	18
3 The Berlin V2X Dataset	21
3.1 Studies using the Berlin V2X Dataset	21
3.2 Dataset Description	22
3.3 Cellular Dataset	25

4	Data Imputation on Berlin V2X Dataset	29
4.1	Data Analysis	29
4.1.1	Types of Missing Data	30
4.1.2	Data Cleaning	31
4.1.3	Feature Engineering	33
4.2	Data Imputation Methodology	35
4.2.1	Data Preparation for Data Imputation	35
4.2.2	Data Imputation Models	36
4.2.3	Evaluation of Data Imputation Model	39
4.2.4	Data Imputation Results	41
5	Data Generation for Berlin V2X Dataset	43
5.1	The concept of synthetic data generation	43
5.2	Data Generation using GANs	44
5.2.1	Generation module for GAN	47
5.2.2	Discriminator module for GAN	48
5.3	Evaluation of Data Generation Model	49
5.4	Data Generation Results	50
5.5	Representation of Real and Synthetic Data	53
6	Conclusion	57
	Bibliography	59

List of Figures

1.1	Thesis Overview	4
2.1	Sensor Technologies in Autonomous Vehicles	6
2.2	Types of V2X Communication	8
2.3	LTE-based Communication	11
2.4	Types of Data in V2X Dataset	11
3.1	Measurement route for Berlin V2X Dataset	23
4.1	Missing Data in Cellular Dataset	32
4.2	DAE Training Loss	37
4.3	GRU-D model architecture	38
4.4	GRU-D Training Loss	39
5.1	Schematic diagram of LSTM architecture	44
5.2	Schematic diagram of GAN architecture	46
5.3	(a) Generator Adversarial Loss and (b) Discriminator Training Loss	49
5.4	LSTM vs GRU-GAN RMSE for High Priority Features	52
5.5	Real vs Generated Feature Distribution	53
5.6	Overlap between the real and synthetic distributions	54
5.7	Data gaps filled using Synthetic Data	55
5.8	Generated route for (a) Device 1 (b) Device 2	56

List of Tables

3.1	Overview of the Berlin V2X cellular dataset	24
3.2	Overview of the Measurement Parameters	25
4.1	Feature groups and average missingness in the Berlin V2X cellular dataset	31
4.2	Temporal data split	35
4.3	DAE architecture and training configuration	37
4.4	GRU-D architecture and training configuration	39
4.5	Feature-wise imputation performance of the GRU-D model	40
4.6	Performance comparison of imputation models	41
4.7	Missingness statistics before and after GRU-D imputation	42
5.1	Identified timestamp gaps for devices pc1 and pc3 on 22 June 2021	43
5.2	LSTM architecture and training configuration	45
5.3	Hierarchical feature grouping for GAN-based synthetic data generation	46
5.4	GRU-GAN generator architecture and training configuration	47
5.5	Discriminator architecture and training configuration	48
5.6	Model comparison between LSTM and GRU-GAN models	51
5.7	Feature-wise evaluation results of the hierarchical GRU-GAN model	52

Abbreviations

Abbreviations	Definition
ITS	Intelligent Transportation Systems
AI	Artificial Intelligence
VANETs	Vehicle ad-hoc Networks
CEN	European Committee for Standardization
ISO	International Standards Organisation
SAE	Society of Automotive Engineers
ETSI	European Telecommunications Standards Institute
ARIB	Association of Radio Industries and Businesses
DSRC	Dedicated Short-Range Communications
V2X	Vehicle-to-Everything
E-UTRAN	Evolved Universal Terrestrial Radio Access Network
LTE	Long Term Evolution
GPS	Global Positioning System
QoS	Quality of Service
ML	Machine Learning
GRU	Gated Recurrent Unit
GAN	Generative Adversarial Network
LiDAR	Light Detection and Ranging
ToF	Time of Flight
IMUs	Inertial Measurement Units
INS	Inertial Navigation Systems
C-ITS	Cooperative Intelligent Transportation Systems
V2V	Vehicle-to-Vehicle
V2I	Vehicle-to-Infrastructure
V2N	Vehicle-to-Network
V2P	Vehicle-to-Pedestrian
OBUs	Onboard Units
RSUs	Roadside Units

Abbreviations	Definition
C-V2X	Cellular Vehicle-to-Everything
MANETs	Mobile Ad Hoc Networks
RAT	Radio Access Technology
BSMs	Basic Safety Messages
IEEE	nstitute of Electrical and Electronics Engineers
WAVE	Wireless Access in Vehile Environment
MAC	Medium Access Control
3GPP	3rd Generation Partnership Project
NR-V2X	New Radio V2X
PC5	Sidelink
eNB	Evolved NodeB
ADAS	Advanced driver-assistance systems
E2E	end-to-end
KPIs	Key Performance Indicators
PQoS	Predictive Quality of Serive
kNN	k-Nearest Neighbors
SVM	Support Vector Machines
ANNs	Artificial Neural Networks
RNNs	Recurrent Neural Networks
LSTM	Long Short Term Memory
RL	Reinforcement Learning
MCAR	Missing Completely At Random
MAR	Missing At Random
MNAR	Missing Not At Random
GRU-D	Gated Recurrent Unit with Decay
DL	Deep Learning
MNOs	mobile network operators
PCell	Primary Cell
SCell	Secondary Cell
DME	Dedicated Measurement Equipment
DL	Downlink

Abbreviations	Definition
UL	Uplink
SDR	Software-defined Radio
SNR	Signal-to-Noise Ratio
RSRP	Reference Signal Received Power
RSRQ	Reference Signal Received Quality
RSSI	Received Signal Strength Indicator
RRC	Radio Resource Control
RBs	Resource Blocks
MCS	Modulation and Coding Schemes
COG	Course over Ground
MCC	Mobile Country Code
MNC	Mobile Network Code
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
DAE	Denoising Autoencoders
MSE	Mean Squared Error

1 Introduction

Recent advances in automobile technology, communication systems, and information technology have transformed traditional vehicles from conventional means of transportation into intelligent, connected, and infotainment-rich computing platforms on the move. These advancements in high-end vehicles, along with improvements in communication and computing technologies, have created a solid base for smart and autonomous vehicles [1]. Also, the progress in wireless communication, embedded systems, sensors, vehicle networking, navigation, data collection, and data analysis are advancing the development of autonomous vehicles [1]. Furthermore, the collaboration of Artificial Intelligence (AI), Machine Learning (ML), and the Internet of Things (IoT) allows modern vehicles to execute complex decision-making tasks with minimal human intervention [2].

These capabilities are derived from the vehicle communication network, which allows vehicles to share data with each other, infrastructure and network platforms which enables cooperative awareness and autonomous driving. To optimise communication systems in the automotive industry, machine learning is gaining significance, particularly for connected and autonomous vehicles. Furthermore, the ongoing standardisation activities for the integration of AI and ML in beyond-5G and future 6G communication networks will accelerate the development of ITS toward more cooperative, data-driven, and autonomous mobility solutions that enable advanced safety, automation, and infotainment applications[3].

The origin of vehicular communication, on the other hand, is far beyond these recent developments. The concept of vehicular communication dates back to 1926 when wireless communication among vehicles was proposed using radio-based warning systems[4]. Other radio services, now regarded as forms of infotainment, include dramas, music or talk shows, also conveyed traffic and safety alerts from public infrastructure through radio broadcasts[5]. These early systems aspired to provide safety benefits comparable to those sought by current vehicle communication systems[5].

In the years that followed, several early advancements in inter-vehicle communication occurred[5]. In order to specify architecture as well as infrastructure of ITS, International Organization for Standardization (ISO) and European Committee for Standardization (CEN) formed technical committees dedicated to ITS in 1991 and 1992[6]. Later, important progress in setting up Vehicle ad-hoc Networks (VANETs) happened when standard groups like CEN, Society of Automotive Engineers (SAE), European Telecommunications Standards Institute (ETSI),

Association of Radio Industries and Businesses (ARIB), and ISO began working on Dedicated Short-Range Communications (DSRC)[5]. Several wireless technologies advanced quickly while the DSRC standard steadily transformed from a theoretical concept to working prototypes[5]. The development of mobile cellular networks as well as the rapid expansion of Wi-Fi technologies became particularly significant in the framework of V2X communication[5].

When the DSRC standardization process was initialized, mobile networks were unable to satisfy the stringent standards of V2X communication[5]. Although there has been a lot of evolution in this regard over the past 20 years, the arrival of Evolved Universal Terrestrial Radio Access Network (E-UTRAN), known as Long Term Evolution (LTE)[7], cellular networks with enhanced uplink and downlink speed, reduced latency, and increased reliability have become widely accessible[5].

LTE technology is already extensively available and enables communication for a significant number of mobile phones and cellular device users[5]. Numerous on-board services in vehicles currently make advantage of cellular connectivity, typically through telemetry systems[5]. These systems not only support Global Positioning Systems (GPS) and other long-range technology but are also often used for infotainment, navigation, and managing vehicle fleets[5].

The emergence of vehicular communication technologies has led to the generation of large amounts of heterogeneous data originating from vehicles, communication networks, sensors, and surrounding infrastructure[8]. Current communication systems are constantly exchanging real-time data about traffic status, wireless network conditions, vehicle mobility, and Quality of Service (QoS) metrics[9]. However, due to the extremely dynamic nature of vehicular contexts, maintaining reliable and comprehensive communication measures is still a considerable difficulty, especially during real-world data measurement operation[9].

1.1 Problem Statement

Vehicular communication is a data-intensive field in which large amounts of measurement data are generated under highly dynamic real-world conditions. Cooperative and connected vehicle systems continuously exchange information related to network quality, vehicle mobility, traffic conditions, and communication performance. As a result, handling and processing this data under real-time conditions becomes one of the most important tasks in vehicle communication network.

However, due to the dynamic nature of connected vehicle measurements, problems such as data degradation, missing values, and data loss can frequently occur. During measurement campaigns

or real-world operation, factors such as connectivity interruptions, unstable wireless conditions, hardware limitations, and measurement failures can lead to incomplete data measurements. Hence, maintaining continuous or consistent measurement data is not always practical or cost-effective.

These missing or degraded measurements present serious challenges for future investigation. As vehicular communication data is highly dense, heterogeneous, and strongly dependent on temporal and spatial relationships, conventional statistical approaches may not sufficiently capture these complex dependencies, especially in highly dynamic vehicular environments. Therefore, advanced techniques are needed that can reconstruct the missing measurements while maintaining the original temporal behaviour and data structure.

The use of Machine Learning (ML) methods overcomes limitations associated with missing measurements and data scarcity. However, application of ML methods on vehicle communication datasets possess challenges such as temporal consistency, feature dependency and realistic data generation for missing time intervals. Thus, the objective of this thesis is to overcome these limitations by proposing methods for data imputation and synthetic data generation of vehicle communication datasets.

The primary aim of this thesis is to apply machine learning methods to fill measurement data gaps, reconstruct missing values, and generate realistic synthetic data for timestamps with missing measurements. To achieve this objective, two deep learning-based approaches are developed. A Gated Recurrent Unit (GRU)-based model is implemented for missing data imputation, while a Generative Adversarial Network (GAN)-based approach is used for synthetic data generation.

1.2 Overview of Thesis

This section provides an overview of the thesis structure as illustrated in Figure 1.1. A brief description of each chapter is presented as follows:

Chapter 2 presents vehicle sensor technologies, background of vehicle communication systems, V2X technologies, and the requirements to generate V2X datasets. It addresses the incorporation of ML in vehicle communication data and the current V2X Datasets. In addition, this chapter introduces the fundamental concepts of data imputation and synthetic data generation in the context of V2X communication.

Chapter 3 describes the selected vehicular communication dataset, namely the Berlin V2X dataset. This chapter describes the studies based on the Berlin V2X dataset, explains the measurement campaign conducted to develop the dataset and presents the two main types of recorded data. It further provides a detailed description of the selected cellular measurement

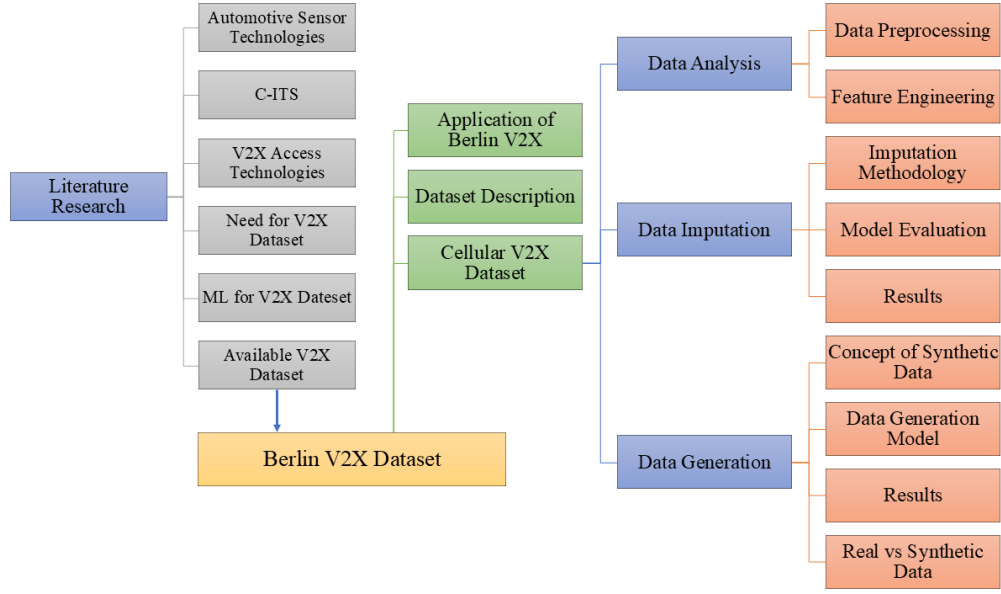


Fig. 1.1: Thesis Overview

dataset, which is used for the implementation and evaluation in this thesis.

Chapter 4 is about the methodology and implementation of the data imputation methods. The implementation is split into two main sections: detailed data analysis, including data preprocessing, and implementation of data imputation techniques. This chapter presents model architectures, training processes, evaluation methods, and outcomes of the data imputation process.

The implementation process of synthetic data generation for the Berlin V2X dataset is presented in Chapter 5. The imputed dataset is used as the input for the synthetic data generation process. The chapter also evaluates the generation model by comparing the statistical distribution of generated data with that of the real data. In addition, it demonstrates the practical application of the model by filling timestamp gaps identified in the cellular dataset. Finally, the generated GPS data are validated by plotting the synthetic data points on the route map.

The thesis concludes with Chapter 6 which summarises the methods implemented, the key results, limitations and overall accomplishments. Future scope of the research is also discussed, and possible improvements and extensions to the proposed approach are highlighted, which include additional methodologies for synthetic data generation and real-time data reconstruction.

2 Literature Research

One of the most important and revolutionary developments in modern transportation is autonomous driving, which has the potential to significantly reduce traffic accidents, improve fuel efficiency, and enhance mobility services. The modern day intelligent vehicles shall be able to observe, interpret, and react to their surroundings in real time since they operate in extremely complex and dynamic situations. These capabilities are primarily enabled by sensor technologies used in autonomous vehicles, which comprises Light Detection and Ranging (LiDAR), cameras, radars, ultrasonic sensors, GPS, and inertial measurement units (IMUs), as shown in Figure 2.1.[10]

- **LiDAR:** LiDAR is a technology for remote sensing which is used to sense the surroundings and measure distances[11]. It is illuminated by active sensors that produce their own light[11]. LiDAR works on the Time of Flight (ToF) principle, using pulsed laser light to measure the time it takes for reflected signals to return[11]. Such measurements are then used to create intricate three dimensional depictions of the surroundings[11]. LiDAR sensors are typically categorised as long range sensors, having detection ranges up to 250 meters. They can measure distances at frequencies more than 150 kHz (150,000 pulse per second)[12].
- **Radar:** Radar is a sensor technology that measures the object distance, angle, and speed using radio waves[11]. It can work in a variety of frequency ranges, including 24 GHz, 77 GHz, and 79 GHz, and is based on electromagnetic radiation[11]. With typical detection ranges ranging from 50 to 100 meters, radar sensors are typically classified as short to medium range sensors, while certain radar systems can identify things more than 150 meters away[13].
- **Camera:** Cameras operate using passive light sensors to capture digital images of the environment. They can detect both stationary and dynamic objects within their field of view. Cameras offer a key advantage over other sensors by recognizing colors and textures. This ability greatly improves autonomous vehicle perception, allowing for accurate detection of road signs, traffic lights, lane markings, and other critical visual cues.[11]
- **Ultrasonic sensors:** An ultrasonic sensor uses sound waves to determine the existence of an object in the path. It measures the distance by sending out sound waves at a particular

frequency and measure how long it takes for reflected waves to return. For close range applications, ultrasonic sensors are considered extremely accurate. However, disruptions in sound waves can have a big impact on their performance.[11]

- GPS: GPS is a satellite based radio navigation system that, if there exists a direct line of sight to at least four satellites, may deliver geolocation and time based information to receivers anywhere on Earth. As GPS provides accurate location data, it is often used for routing and positioning in autonomous cars. Commercial grade GPS systems can provide location precision within about 1 m, whereas standard hand-held GPS units that use single frequency signals usually achieve accuracy of about 3 m.[11]
- IMUs: An Inertial Measurement Unit is an electronic equipment that measures angular velocity, acceleration, and magnetic field data. It typically consists of gyroscopes, accelerometers, and magnetometers aligned along the orthogonal X, Y, and Z axes. IMUs are frequently utilised in autonomous cars for guiding and control purposes. They are frequently integrated with Inertial Navigation Systems (INS), which estimate parameters such as linear velocity, orientation, and angular position relative to a global reference frame by processing raw sensor data.[11]

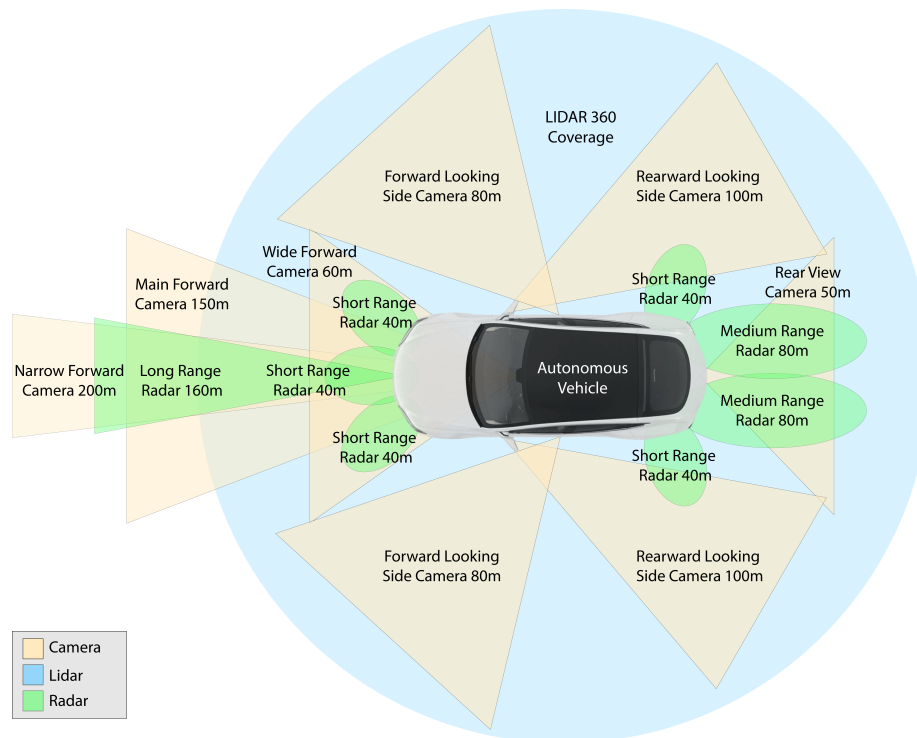


Fig. 2.1: Sensor Technologies in Autonomous Vehicles[14]

Although the above mentioned onboard sensors can handle a wide range of driving scenarios, their reliability is limited by sensing range, field of view, environmental conditions, and occlusions. As a result, the understanding of vehicle for the complete traffic environment is often limited by the range of these sensors. This limitation creates the need for information exchange among vehicles, road infrastructure, and communication networks, forming the basis of Cooperative Intelligent Transportation Systems (C-ITS).

2.1 Cooperative Intelligent Transportation Systems

Research in C-ITS communication has achieved substantial advances to improve traffic control, in-car infotainment, autonomous driving, and important road safety tasks[5]. To achieve these capabilities, coordination across multiple organisations and support for multiple forms of communication is needed[5]. These forms are vehicle-to-vehicle (V2V), vehicle-to-infrastructure (V2I), vehicle-to-network (V2N), are generally known as V2X communication[5].

V2X communication is one of the ITS domains and involves wireless communication and coordination between vehicles and their surrounding environment. Briefly, ITS is a term to describe networked systems which are developed to offer safer and more efficient transportation to the users. ITS covers systems in the air, rail, and maritime industries, as well as in road transport. ITS applications are equally diverse and include basic alert and warning systems, infrastructure level synchronization, onboard driving support, and vehicle-to-vehicle communication[5].

- The term V2V communication describes the direct message exchange between cars, usually via an ad hoc, self-organised network without centralised management. These communications are sent between vehicles that have V2X equipped onboard units (OBUs).[15]
- V2I communication involves the exchange of data between Roadside Units (RSUs) and vehicles. RSUs are electronic devices that can receive, process, and send communication messages between automobiles and other transportation infrastructure elements. V2I communication can take place via centrally managed or ad hoc networks.[15]
- Vehicle-to-Pedestrian (V2P) communication is the transmission of messages between cars and wearable or handheld devices with V2X capabilities that are used by bicycles, pedestrians, and other road users. Infrastructure networks that are centrally administered, like LTE or 5G cellular systems, typically facilitate this kind of communication.[15]

- V2N communication is connected with cellular networks, allowing vehicles to communicate with other ITS components, such as V2X management systems and application servers, via existing cellular infrastructure.[15]

A schematic diagram illustrating V2X communication and its different communication modes are shown in Figure 2.2

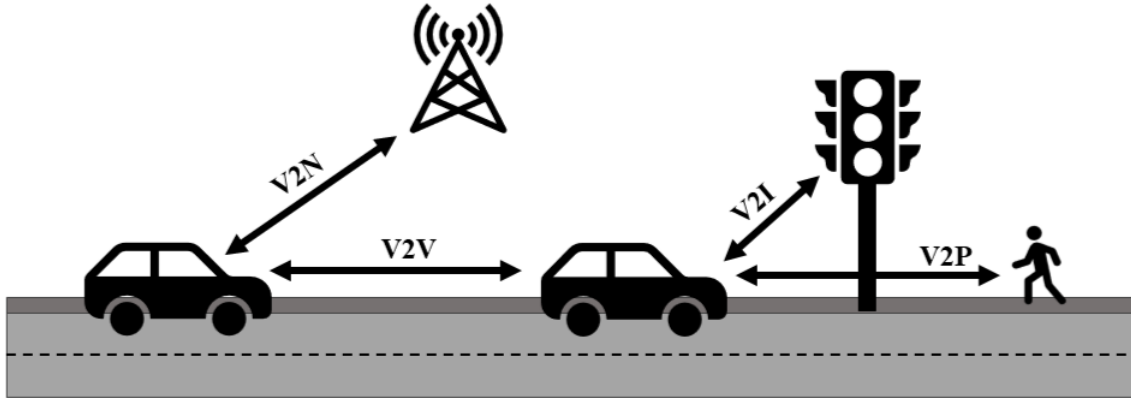


Fig. 2.2: Types of V2X Communication

While V2X offers a wide range of applications including traffic safety, traffic efficiency, infotainment, and cooperative driving[5]; several major challenges remain:

1. **Standardization:** Standardization enables compatibility and interaction among different systems and components which enables efficient communication between vehicles, infrastructure, and related equipment[16]. It also enables the greater adoption and widespread application of these technologies while reducing costs and minimising risks associated with development, deployment and operation [17].
2. **Security:** V2X technology encompasses communication among vehicles and infrastructures; any vulnerability may compromise the safety of passengers and vehicle operation; therefore, security is one of the vital components. Among the primary security challenges are secure communication routes, validation and authorisation for devices, protection of sensitive data, and protection from potential cyberattacks.[16]
3. **Cost:** The expensive cost of V2X technology is one of the biggest obstacles to its deployment. A substantial expenditure in sensor technology, communication systems, and other software and hardware components is required to set up the required infrastructure. Future renovations and continuing maintenance costs must also be considered.[16]

4. **Reliability:** One of the main issues with V2X technology is reliability. To guarantee that safety-critical communications are transmitted on time, especially for applications like collision prevention and emergency warnings, the communication network must be highly reliable. The framework must be built to reduce interference, delays, and message loss in order to do so.[16]
5. **Integration:** To achieve its full potential, V2X technology is required to be combined with other technologies, such as autonomous vehicles and smart city solutions. To guarantee smooth and successful integration, strong cooperation between many sectors and industries is necessary.[16]

2.2 V2X Access Technologies

C-ITS leverages wireless connectivity and real-time data exchange to enable intelligent, cooperative interactions among vehicles, infrastructure, pedestrians, and other road users[18]. In this context, V2X communication improves cooperation, potentially preventing significant amount of current road accidents and advancing the automobile and telecommunication industries towards safer and smarter transport systems[19]. To realize V2X communication, two main technological approaches have been widely adopted: DSRC and Cellular Vehicle-to-Everything (C-V2X)[19].

2.2.1 Dedicated Short Range Communication

VANETs have emerged as a major area of interest for enabling V2X communication, building upon research in Mobile Ad Hoc Networks (MANETs) [5]. V2X communications have relied on DSRC based on the IEEE 802.11p standard as a core Radio Access Technology built particularly for automotive and ITS applications[20]. DSRC enables nearby units to exchange status information via short range radio communication[16]. This information is known as Basic Safety Messages (BSMs) and includes important vehicle data such as speed, location and direction[16].

The DSRC system is defined through a set of standards developed by the Institute of Electrical and Electronics Engineers (IEEE) and SAE International which specify the network architecture and security mechanisms[16]. In the US, the term "DSRC" is often used for the Wireless Access in Vehicle Environment (WAVE)[21] technology, while in Europe it is more often used for earlier ITS systems. The technology is mostly based on IEEE 1609 family of standards and the IEEE 802.11p standards (based on IEEE 802.11a). In particular, the IEEE 802.11p standard establishes both the physical and Medium Access Control (MAC) layer architecture for DSRC[16]. This

framework simplifies communication under highly dynamic channel conditions and verification before data transmission. It enables vehicles to directly broadcast safety related information to nearby units, supporting fast and localized data exchange[16]. In practice, DSRC based communication operates through the interaction between in-vehicle wireless transmitters known as OBUs, and RSUs deployed along transportation infrastructure [5].

DSRC technology provides a number of benefits for vehicular communication such as low latency data transmission, robust communication including real-time data sharing between vehicles and the infrastructure[16]. These features are helpful in developing numerous automotive applications such as traffic management, collision prevention, cooperative awareness and crossroad safety[16]. However, DSRC has several drawbacks such as limited bandwidth and sensitivity to interference which can degrade its performance in dense traffic environments and limit its applicability to high bandwidth applications[16].

Therefore, while DSRC has played an important role in early V2X development, its limitations have motivated research into its successor IEEE 802.11bd. The competitive alternative to DSRC technology is C-V2X communication as described further.

2.2.2 Cellular V2X Communication

V2X technology based on cellular network standards, also referred to as Cellular V2X, is another potential V2X access technology[5]. C-V2X is described in 3rd Generation Partnership Project (3GPP) Release 14 specifications and it refers to the 4G-based implementation of the technology while, the 5G-based C-V2X framework outlined in the relevant 3GPP Release 16 specifications[15] which are also known as New Radio V2X (NR-V2X). The LTE-V2X (4G based V2X) as well as 5G-NR-V2X technology uses both the Uu interface and sidelink(PC5) interface to send and receive messages, providing important communication for facilitating autonomous driving[18]. The Uu interface promotes wide range communication between a base station and a user device, while the PC5 communication allows for direct shorter-range V2V / V2I communication[22].

The Figure 2.3 illustrates two modes of communication in the C-V2X networks. These networks offer several advantages over DSRC including wider coverage, the use of existing infrastructure, more reliable safety and QoS support, and improved scalability [5]. These characteristics make cellular technology highly suitable for facilitating V2X communication particularly because of its high network capacity for bandwidth intensive and data demanding applications and its technological maturity, which simplifies implementation and accelerates deployment[19].

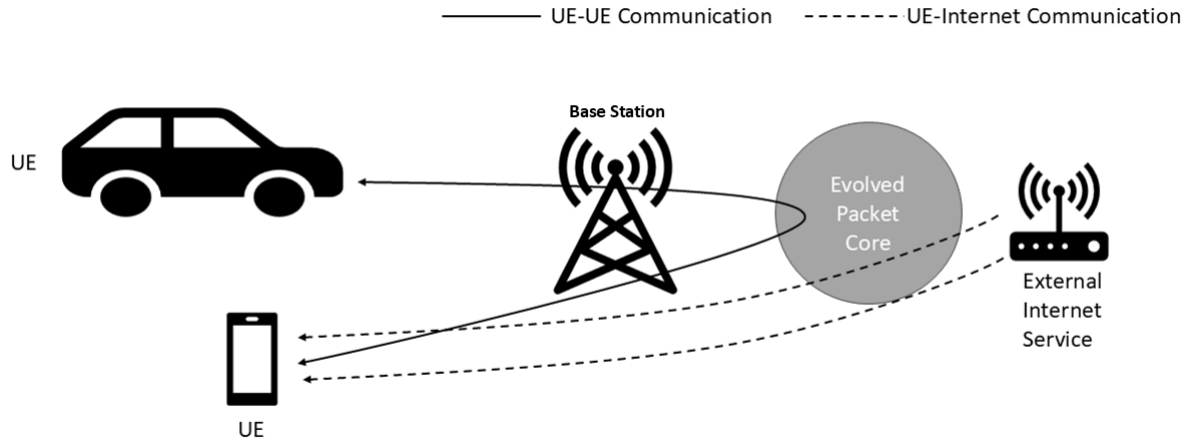


Fig. 2.3: LTE-based Communication[5]

2.3 Need for V2X Dataset

The quality of the underlying data determines the performance of vehicular communication systems. Vehicle position, speed, direction, environmental conditions, and network parameters must be accurate, reliable, and available in real-time to support safety critical and autonomous driving applications [23]. The datasets used during the assessment and development of vehicular communication systems form the foundation for training machine learning models that support decision making processes in V2V, V2I, and in-vehicle communication systems[24]. Therefore, selecting an appropriate dataset is a challenging task that requires assessing elements including data structure, quality of representation, scalability, and the degree to which the data correctly reflects realworld conditions.[24]

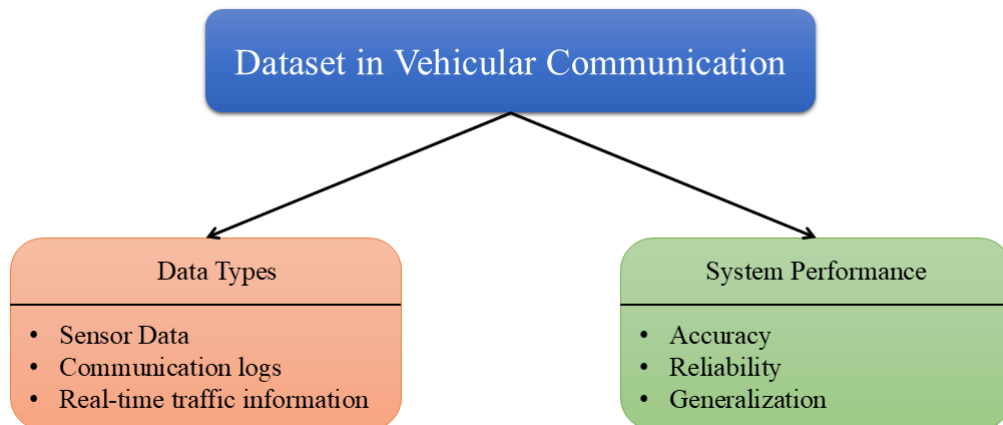


Fig. 2.4: Types of Data in V2X Dataset[24]

The selection of datasets containing high quality realtime measurements is essential for reliable V2X data analysis and machine learning models. A typical V2X dataset could consists of three main categories of data:

1. **Sensor Data:** Sensor data offers real-time information from the environment of the vehicle. Hence, it is an important part of the vehicular communication system. LiDAR, cameras and other sensors collect data by perceiving information from the environment. This data can be used for tasks such as navigation, obstacle avoidance and object detection. Sensor data are heavily used in advanced driver assistance systems (ADAS) and autonomous driving systems to operate vehicles safely and efficiently in adverse conditions.[24]
2. **Traffic Data:** Realtime traffic data plays an essential role in V2I communication; this includes communication of vehicles with traffic control systems and responding to road conditions. The data includes traffic movement, traffic jam levels, road restrictions and weather information. This information can assist cars to plan better routes, save travel time and fuel consumption, and make traffic management safe and effective.[24]
3. **Communication Data:** The communication logs are a crucial part of a V2X dataset. They provide valuable information for determining the reliability and performance of V2V and V2I networks. These logs are records of data exchanges between vehicles and infrastructure through communication systems (DSRC or cellular). They include features such as message time, frequency, and message type. Additionally, they contain data like timestamps, transmission rates of data, jitter, latency, etc. that provide a thorough picture of the way communication protocols function in different situations.[24]

The main aim of this thesis is to focus on communication data, since it plays an important role in measuring network performance, identifying shortcomings and improving the performance of vehicle communication systems. Hence, data collected from different sources shall be used to derive meaningful conclusions about the accuracy, reliability, and generalisation of specific behaviours in vehicular systems as shown in Figure 2.4. As Machine learning is a data-driven approach, it can be a powerful tool for studying, analysing, and drawing conclusions from high quality datasets.

2.4 Machine Learning for V2X Data

Machine learning offers a variety of methods for the analysis and extraction of findings from diverse sources of data generated in the context of vehicular networks. This enables systems

to cope with communication problems, make data-driven decisions, and support sophisticated applications like realtime traffic forecasting and control as well as automated driving.[24]

V2X communications can be enhanced using ML approaches towards achieving intelligent and adaptive decision making in highly dynamic vehicular scenarios. ML approaches are often used for optimisation of network services such as handover management[25], resource allocation[26], routing[27] and QoS[28]. ML models can process large volumes of vehicle and network data to predict network conditions, improve communication reliability, reduce latency, and increase traffic efficiency and safety.[8]

2.4.1 Application of Machine Learning in V2X

This section presents the applications of ML for the above-mentioned V2X communication services:

- **Handover Management:** Handover management is a critical aspect in wireless networks to provide uninterrupted service as User Equipment moves around nearby cells. Handover is the transfer of a serving base station from one cell to another based on defined parameters such as radio signal thresholds. Handovers can affect end user overall quality of service because they are highly dependent on radio circumstances. Predictive machine learning is used to improve base station discovery rates, reduce needless handovers, and improve handover decision-making.[8]
- **Resource allocation:** Radio Access Network(RAN) resource allocation is the control of power, spectrum, and channel access in order to reduce user disturbance and improve overall network throughput. This allocation can be done in radio networks utilising either distributed architectures where vehicle users take part in the allocation process or centralised architectures, where base stations decide how to allocate resources. Machine Learning helps with this endeavour by optimising network resource distribution in response to dynamically changing traffic and connection requirements.[8]
- **Routing:** One of the most difficult aspects of vehicular networks is efficient data distribution among vehicles which requires choosing appropriate relay nodes and improving multi-hop routing methods. As the density of vehicles continues to rise in future networks, the primary goal is to guarantee low latency and good dependability. By identifying appropriate forwarding techniques, machine learning can enhance path-selection evaluations in dynamic vehicular topologies.[8]

- **QoS Predictions:** Cooperative self-driving systems require strict QoS assurances, especially in terms of data rate, reliability and end-to-end (E2E) latency[28]. However, the high dynamic nature of vehicular networks leads to rapid variations of key performance indicators (KPIs) over time[28]. These rapid variations make it difficult to reliably meet the communication requirements needed for secure autonomous driving[28]. Hence, realtime QoS prediction is critical for ensuring robust autonomous systems by enabling the network to anticipate degradations before they occur [28]. Cooperative driving systems can proactively adapt their operation to degraded communication performance by predicting future network conditions[28]. To address this, Machine Learning approaches have been used to classify E2E packet delay with respect to given thresholds, predict future latency trends, determine whether a packet can be delivered within a latency requirement and predict QoS user requirements to activate or deactivate dual connectivity[8].

2.4.2 Machine Learning Methods for V2X Data

According to the application and network requirements, the ML approaches used in V2X data processing and automotive communication systems are classified as:

1. **Supervised Learning:** In supervised learning, algorithms are trained on input data and their corresponding output, called features and labels. The labels specify the type of problem: for regression problem the output is continuous whereas for classification it is discrete. The model learns the relationship between the features and the labels through a target function to make accurate predictions.[8]

Hence, prediction and classification tasks in V2X networks are performed by utilising supervised learning methods. These approaches learn from labelled datasets to predict future outcomes such as signal strength, traffic flow, vehicle trajectory and QoS levels[8]. Some of the widely used supervised ML algorithms are k-Nearest Neighbours (kNN)[29], Support Vector Machines[30], Artificial Neural Networks (ANNs)[31], Recurrent Neural Networks (RNNs)[32] and Long Short Term Memory (LSTM) models[32].

2. **Unsupervised Learning:** Unsupervised learning refers to the process of training models using only unlabelled data by pattern identification within data without prior knowledge of output categories.[8]. The main tasks in this approach are clustering, which groups similar data points; dimensionality reduction, which decreases input features while retaining essential information; and density estimation, which models the underlying probability

distribution of the data[8].

As vehicular environments generate vast amounts of heterogeneous data, unsupervised learning approaches are effective for identifying traffic flows, clustering similar vehicle behaviours, detecting network anomalies, and increasing overall communication efficiency[8]. Some unsupervised learning techniques applied in V2X systems include K-means clustering[33], K-medoids[34], and affinity propagation[35],[8].

3. Reinforcement Learning (RL): It is a subfield of machine learning that focuses on making optimal decisions through interaction with the environment[8]. An agent observes the environment, takes actions, and receives feedback in the form of rewards or punishments[8]. The objective is to maximize cumulative rewards over time by developing an effective policy[8]. As opposed to supervised learning, this approach does not use labeled data; rather, it enhances performance through feedback driven encounters with the environment[8].

The suitability of a machine learning approach for V2X systems is determined by the dataset structure, the availability of labelled data and the specific task to be performed. Therefore, no single ML strategy can be considered universally applicable across all V2X related situations, as different objectives require distinct modelling approaches and learning strategies.

The main emphasis of this thesis is to use ML approaches for data imputation and synthetic data generation using V2X datasets. Data imputation is employed to estimate and reconstruct missing or incomplete values within the dataset, while data generation techniques are used to produce realistic synthetic vehicular data samples that preserve the statistical characteristics and patterns of the original dataset.

2.5 Data Imputation and Data Generation using ML

2.5.1 Data Imputation

During measurement campaigns, vehicles are constantly moving with changing traffic conditions, road layouts and wireless channel environments. This means that the recorded data is often affected by signal fluctuations, variations in network coverage, and interruptions in the measurement process at certain points. These factors can result in missing or incomplete values in the dataset, leading to inconsistencies and reducing the reliability of the dataset for further analysis and model development. The efficient management of missing data is a significant barrier to V2X data analysis, since repeated measurement campaigns are expensive, time consuming, and

difficult to conduct on a regular basis.

There are two main techniques to handle missing data in datasets. One way is to remove all incomplete samples, but this can lead to losing important information and may change the original distribution of the data.[36]. It can also have a detrimental impact on the efficiency of downstream models, particularly when analysing traffic data, and high missing rates can lead to inadequate data for analysis[36]. The second method is data imputation. This strategy predicts missing data by identifying relationships and patterns in available data leading to better dataset integrity and quality [36]. In most cases, this method is more efficient and preferable to discarding incomplete samples. Statistical patterns of missing data are often classified into three forms depending on the degree of randomness[37]:

- Missing Completely At Random (MCAR): This term describes situations where missing information is unrelated to any other variables, indicating that the possibility of missing data is completely random and independent of observed as well as unobserved variables[36].
- Missing At Random (MAR): It suggests that the missing information is largely random, implying that its occurrence is linked to specific observed variables. The missingness has a certain pattern or explanation, even if it has no direct connection to the absent value itself.[36]
- Missing Not At Random (MNAR): It signifies that the missing information has a direct relationship with the value of variable. In this situation, the missing values are difficult to infer based on other data available and most existing strategies for dealing with missing data struggle to handle this sort of missingness.[36]

These patterns describe the relationship between the probability of missing data and the observed variables. Therefore, it is important to examine these relationships while dealing with missing data for appropriate analysis and modelling.

There are many machine learning based algorithms which are commonly used for missing data imputation. kNN is a popular baseline approach among them[38]. kNN approaches locate (k) samples in the dataset with temporal or spatial similarity via distance measurements[36]. The values of the missing data points are then estimated using these nearby samples[36]. The effectiveness of this approach is strongly dependent upon parameter selection and identifying the ideal value of (k) continues to be a tough task, adding complexity in actual applications [38].

To capture complex correlations and dependencies within the data, neural network based models have been introduced for missing data imputation [39]. One of the neural network model

used for missing data imputation is denoising Autoencoder [39]. The Denoising Autoencoder (DAE), which was first presented by Vincent et al. [40], is taught to reconstruct clean input data from purposefully contaminated inputs. The model learns to retrieve the original values by randomly masking a fraction of observed values (for example, 20%) during training[39].

Since V2X data is collected over time, it can be effectively treated as time-series data[36]. The above mentioned imputation techniques often struggle to capture complex temporal dependencies in highly dynamic V2X environments. Therefore, advanced machine learning approaches that incorporate temporal dependencies on data shall be considered. RNNs are neural networks containing memory that can detect temporal patterns in sequential input[41]. Their main advantage is that they can capture interdependencies and contextual information by consistently updating hidden states which makes them perfect for predicting missing values based on historical data[36]. Therefore, deep learning based sequential models like LSTM and GRU have attracted much attention due to their ability to model nonlinear temporal relationships and long-term dependencies in time-series data[36].

1. LSTM-based: LSTM networks are a type of RNNs that can learn long-term dependencies in sequenced data by using gating techniques to limit the flow of information[36]. The LSTM-based model is ideal dealing with sequential data and successfully captures temporal trends in traffic data[36]. However, in most cases, spatial interactions must be captured using a combination of methods[36].
2. GRU-based: GRU is another type of RNNs that uses gated mechanisms to regulate information flow. They have the advantage of employing fewer parameters and operating similar to LSTM models. Furthermore, [42] introduced the GRU-D model that incorporates both masking vectors and time interval information that allow the network to learn informative missingness patterns while preserving temporal dependencies in irregularly sampled timeseries data.[36]

Hence, considering the advantages of the above frameworks, it is evident that deep learning based approaches are highly suitable for data imputation in V2X communication systems.

2.5.2 Data Generation

Data generation in a dataset is primarily used to compensate for missing or unavailable data resulting from failures during the measurement period, including equipment malfunction, hardware/software errors, or other issues affecting data acquisition. Moreover, it can also be used to

predict or generate data for time frames beyond the original measurement period. Considering the purpose of this thesis topic, the main objective of synthetic data generation is to fill gaps in measured data using ML or Deep Learning (DL) frameworks. For vehicular communication data, it is critical to evaluate the feasibility of incorporating vehicular data into DL frameworks, as these models generate predictions by identifying underlying patterns and correlations in input data [43].

In [43], a hybrid deep learning framework for synthetic data generation was proposed, combining LSTM networks and GANs to estimate performance indicators in V2X communication systems.

One of the most widely used approaches for synthetic data generation is GAN, which frames the data generation process as a supervised learning problem [44]. A GAN consists of two neural networks: a generator module and a discriminator module[45]. The generator aims to produce data that closely resembles authentic data, while the discriminator is responsible for distinguishing between real and synthetically generated data[45]. This is simply a binary classification problem in which genuine samples are evaluated as positive and produced samples as negative [45]. Both networks are trained at the same time in a minimax framework where the generator attempts to deceive the discriminator, while the discriminator attempts to distinguish real from fake input[45].

As a result, this hybrid approach is effective in maintaining the temporal dependencies of the data as well as maintaining statistical characteristics of data as implemented in [45], which has been used as the motivation for the generation of synthetic data where the detailed description of the model architecture and methodology has been discussed.

2.6 Available V2X Dataset

Several open-source V2X datasets have been introduced in recent years to support research in connected and automated driving. However, these datasets differ significantly in terms of their objective, data modality, measurement environment, and suitability for machine-learning tasks related to wireless communication. Therefore, before selecting the dataset for this thesis, available V2X datasets were analyzed with respect to their relevance for communication data reconstruction, missing-data imputation, and synthetic data generation.

Many available V2X datasets mainly focus on cooperative perception. For Example, To address real-world issues in V2X collaboration, the open-source simulation dataset V2XSet[46] was created with CARLA[47] and OpenCDA[48]. The dataset offers almost 11,000 frames

including V2X cooperation data as well as realistic noise simulation, however focusing mostly on junction scenarios[24]. V2XSet consists of 32-channel LiDAR sensors deployed on infrastructure units and vehicles, collecting data at a frequency of 10 Hz[24]. The dataset is primarily used to study the effects of temporal and spatial uncertainties on 3D object detection accuracy [24].

To enable the study of multi-agent collaborative perception in autonomous driving based on the CARLA-Simulation of Urban Mobility co-simulation, a dataset named V2X-Sim was developed. The dataset contains multi-agent multi-modal sensor data collected from cars and RSUs in real-world traffic scenarios. It also has a variety of annotations to help with different 3D perception tasks.[49]

TumTraf-V2X[50] is another open-source dataset that was created using actual traffic data. It has approximately 30,000 3D bounding boxes, approximately 5,000 photos, 2,000 labeled point clouds, and accurate GPS and IMUs readings for precise object tracking[24]. The dataset records a variety of traffic situations, including intricate driving techniques like overtaking, U-turns, and moving infractions[24].

Apart from perception focused V2X datasets, only a limited number of open-source datasets are dedicated to communication and network performance measurements. Examples include V2X predictive QoS datasets[51], which provide two datasets V2I and V2V, generated in a realtime environment at two different locations, provide radio channel measurements for QoS and channel state prediction; the BME V2X dataset[52], which contains real-world ITS-G5/802.11p communication measurements collected from multiple vehicles; and CICV5G [53] provides 5G communication delay measurements along with limited network parameters for intelligent connected cars in cloud based systems. These datasets are based on communication analysis but tend to be restricted to particular technologies, scenarios or prediction tasks.

Each of the datasets listed above corresponds to a specific application area in V2X research. V2XSet [46] and V2XSim [49] mainly provide simulation based data for multi-vehicle sensor data collection and cooperative perception. TUMTraf-V2X extends this direction by offering real-world sensor data from ego vehicles and roadside infrastructure for 3D object detection. In the case of communication based datasets, PQoS [51] provides V2I and V2V communication measurements; however, its scope is mainly limited to QoS and channel related data collected under different conditions. Similarly, the BME V2X [52] dataset focuses on specific V2X communication technologies, while CICV5G [53] mainly considers GPS information and selected cellular network parameters.

Therefore, these datasets are either perception-focused, simulation based, or limited to specific communication data. This creates the need for a real-world measurement dataset that combines both vehicular and wireless communication information under dynamic driving conditions. The Berlin V2X dataset[54] fulfils these requirements through a real-world measurement campaign conducted in urban traffic scenarios using four vehicles over three days. It provides high-quality cellular and sidelink communication measurements collected from multiple vehicles in an urban environment. In addition, it includes QoS parameters, radio signal indicators, GPS-based mobility information, traffic data, and weather data. These heterogeneous features make it suitable for studying missing values, signal fluctuations, and time dependent behaviour in V2X communication.

3 The Berlin V2X Dataset

The Berlin V2X[54] dataset represents a comprehensive real-world vehicular communication dataset that provides high resolution measurements of both vehicular mobility and wireless network performance, supporting research in machine learning and QoS prediction.

3.1 Studies using the Berlin V2X Dataset

This dataset was created primarily to fill a significant gap in the available resources. Several studies have used this dataset to investigate different research questions, as discussed below.

For effective V2X communication in 5G communication networks, determining the ideal network nodal positions is crucial [55]. These nodal points indicate the geographic locations where the wireless network offers optimal connectivity and communication performance [56]. Therefore, [56] proposes a technique that uses data rate prediction in wireless communication to find network nodal points for different areas on the route of the Berlin V2X dataset[54]. The goal of this work was to predict datarate by analyzing network and environmental factors such as ping, precipitation intensity, and weather data[56]. Using datarate as a major performance indicator, this approach suggested a Predictive Quality of Service (PQoS) assessment method for V2X communication [56]. Improved QoS performance and increased communication efficiency can be achieved along the entire route by locating area-wise ideal nodal sites [56].

QoS level prediction ensures dependable communication by sending proactive notifications to vehicles when expected QoS performance is not met, leading to prompt corrective actions[57]. This research introduces a supervised regression model to estimate downlink throughput in a C-V2X network [57]. The model is trained on the Berlin V2X dataset [54] and is based on a straightforward implementation of the Light Gradient Boosting Machine (LightGBM)[57].

Another application of the Berlin V2X dataset[54] is the improvement of QoS prediction in urban environments. The study addresses the prediction of the instantaneous downlink throughput by estimating the current or near-future communication state of the ego vehicle. For this purpose it utilises both temporally aligned present data and geographically aligned past data from the lead vehicle. The study aims to determine whether lead-vehicle data can improve the ego vehicle's PQoS prediction in urban scenarios. It also examines whether previous measurements from the lead vehicle at the same location can improve prediction accuracy. In addition, the study analyzes the impact of relative feature engineering on model performance and evaluates whether the benefits of lead vehicle features remain consistent across different machine learning models.[58]

Due to the highly dynamic nature of wireless communication, the available bandwidth is affected by factors such as vehicle motion, interference and fluctuating network conditions. If an application transmits more data than a wireless connection can handle, Transmission buffers might become full, resulting in increased packet loss or delay. This may degrade the overall Quality of Experience (QoE), particularly for latency sensitive applications. To make optimal use of the connection and avoid overloading it, it is important to estimate future bandwidth needs.[59]

In this context, the BandSeer[59] presents an effective bandwidth prediction approach using a stacked Bidirectional LSTM (Bi-LSTM)[60] model. In this study, the importance of feature availability, feature correlation, and feature selection in bandwidth prediction is emphasised through an analysis of several publicly available datasets[59]. The Berlin V2X dataset[54] was shown to be the most appropriate among the evaluated datasets, since the strongly correlated features can lead to more accurate bandwidth prediction than other available datasets[59].

Building upon the above studies, BiLQoS uses deep learning to predict QoS in C-V2X networks[61]. This work provides a lightweight two-layer Bi-LSTM model to predict crucial QoS metrics such as data rate and latency[61]. It trains and assesses the model in real-world cellular vehicle communication scenarios using the Berlin V2X dataset[54],[61]. They use an operator-agnostic training technique to improve the model's applicability to other smart-city scenarios and reduce dependence on a specific network operator[61].

3.2 Dataset Description

While many vehicle datasets focus on mobility traces using GPS and other sensor data, there was a need for a well-curated dataset that combines vehicular and wireless communication data in a format easily accessible to the research community[54]. The creation of datasets with vehicle data and network data requires a significant amount of time, effort, and specialised tools, as data must be collected in a range of driving circumstances, including urban regions, highways, rural routes, or combinations of such conditions[54].

The Berlin V2X dataset contains measurements across numerous vehicular contexts, including urban and highway scenarios. It includes network data from two different mobile network operators (MNOs), allowing the study of the generalisation of machine learning models across several LTE networks. The dataset also contains high resolution statistics for sidelink and cellular communications, as well as rich traffic and weather-related metadata.[54]

The data measurement procedure was carried out through test drives on a predefined route that includes residential streets, avenues, parks, tunnels, and highways in the West Berlin region. The measurement campaign involved four vehicles traveling along a 17.2 km route under two different driving profiles, namely platoon and 2×2 mode, during weekdays as illustrated in Figure 3.1.[54]

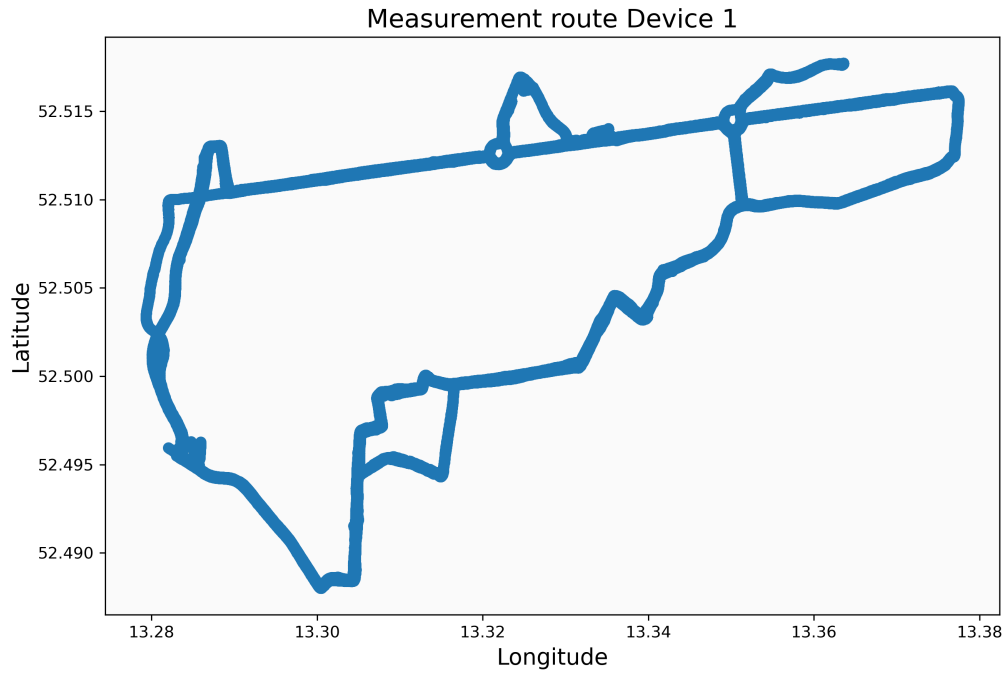


Fig. 3.1: Measurement route for Berlin V2X Dataset

The measurement campaign includes two RATs for the process of data collection:

1. Cellular Communication: Each vehicle is equipped with dedicated measurement equipment (DME) that records key network parameters during interactions with a remote server over LTE. The server was hosted in a data facility in Berlin. Measurements were conducted using two MNOs public LTE networks operating in frequency bands from 700 MHz to 2.7 GHz. Two cars utilised the Deutsche Telekom network, and the other two were linked to the Vodafone network. In this study, the operators shall be referred as Operator 1, Operator 2 to prevent direct comparison.[54]

Several sources, including MobileInsight[62], Tcpdump[63], and Iperf[64], were used to generate traffic and collect data. Both high and low data rates were used in the experiments. While maximum throughput was measured using datarate of higher range (75 Mbit/s

uplink(UL) and 350 Mbit/s downlink(DL)), latency was assessed using low data rate situations (400 kbit/s for both UL and DL). For each test cycle, a particular UL/DL configuration was assigned to each vehicle, and to make data processing easier, transmissions were labelled with distinct port numbers.[54]

Tab. 3.1: Overview of the Berlin V2X cellular dataset[65]

Property	Detail
Measurement devices	4 (pc1, pc2, pc3, pc4)
No. of measurement runs	17 clockwise rounds, each of 17.2 km
No. of Days of measurement	3
Regions of measurement	residential streets, avenues, parks, tunnels, and highway
Drive mode	Platoon, 2 x 2
Data sampling rate	1 second
Dataset shape	207,434 rows x 159 columns
Network type	LTE (4G) [2 Operators] – Primary Cell (PCell) + Secondary Cell (SCell)
Datarate range	Throughput (75 Mbps UL/ 350Mbps DL) , Latency (400Kbps UL/DL)

2. Sidelink Communication: Each vehicle was outfitted with a software-defined radio (SDR) platform that served as sidelink user equipment in accordance with 3GPP Release 14 specifications, in addition to cellular measurements[54]. The sidelink communication data was collected based on two types of messages:

- Cooperative Awareness Messages (CAM): These are periodic messages sent by vehicles to provide real-time status information to the nearby vehicle or road infrastructure. These messages usually contain information of vehicle position, speed, direction of travel, acceleration, types of vehicle etc. Such messages increase the situational awareness among vehicles and RSUs for purpose of safety and traffic efficiency.[54]
- Cooperative Perception Message (CPM): These messages allow vehicles or RSUs to share sensor data (for example, data from LiDAR, radar, camera) to other units. These messages provide information about position and classification of objects like pedestrians, bicycles and other vehicles which help in enhancing environmental awareness beyond line of sight.[54]

Table 3.2 below summarises the numerous types of data collected during the measurement campaign along with their sources and sampling intervals. It shows the vehicular communication

V2V and wireless network communication V2X data in the dataset, demonstrating the heterogeneity of the Berlin V2X dataset[54]. Therefore, the dataset is divided into two categories: sidelink data and cellular data. From the perspective of this thesis, the implementation is restricted to the cellular communication dataset. However, the proposed methodologies are not limited to cellular measurements and may be extended to the sidelink communication dataset.

Tab. 3.2: Overview of the Measurement Parameters[54]

Data category	Source	Tool	Sampling interval	Features
Cellular	DMEs	MobileInsight	Event-based	Radio resource control (RRC): cell identity, DL/UL frequency, DL/UL bandwidth
			10 ms	Physical layer (PHY): SNR, RSRP, RSRQ, RSSI
			20 ms	Physical shared channels: Resource Blocks, transport block size, MCS
	Server	Ping	1 s	Delay
		Iperf	1 s	DL datarate, jitter
		Iperf	1 s	UL datarate, jitter
Sidelink	SDR platforms	TCPdump	Event-based	SNR, RSRP, RSRQ, RSSI, noise power, RX power, RX gain
Position	GPS	–	1 s	Latitude, longitude, altitude, velocity, heading
Side information	Internet database	HERE API	5 min	Traffic jam factor
		DarkSky	1 h	Cloud cover, humidity, precipitation intensity/probability, temperature, wind speed

3.3 Cellular Dataset

The cellular dataset is a subset of the Berlin V2X dataset[54], representing real-world measurements of vehicular communication over cellular networks. It includes diverse set of data such as network parameters, GPS based information, weather conditions and also essential QoS metrics like datarate, latency, jitter etc. The combination of communication, location and environmental data provides a comprehensive overview of network performance in dynamic urban driving scenarios.

Each feature in the dataset can be classified according to its source, output type, and

underlying data properties. The dataset contains various types of data which have been divided into following categories:

1. GPS data: These features are used to represent the position and real-time mobility features of the vehicle which are defined as:
 - Latitude and Longitude: Geographic position of device on earth
 - Altitude: Height of the device above mean sea level
 - Speed: speed of the vehicle during measurement in km/hr
 - COG (Course over Ground): Direction of travel at 0° North. This feature is used to determine the Direction of Movement of the vehicle in a particular lane.
 - Position in Ref round: Position of vehicle in the predefined test loop. This feature is a derived feature from the Latitude and Longitude of the vehicle from an arbitrary reference point in the test loop.
 - Area: It represents the geographic location of the vehicle in the test loop (parks, avenues, highways, residential streets etc.)
2. Weather and Environmental Data: These features indicate information about ambient conditions which are extracted from internet database.
 - temperature/apperentTemperature: actual/feels like temperature at the time of measurement.
 - precipProbability/precipIntensity: These features represent probabiltiy and rate of rainfall/snowfall during the time of measurement.
 - dewPoint, humidity, windSpeed, cloudCover, uvIndex, visibility: These features describe environmental conditions that may affect the accuracy of the network measurements. These factors can influence the quality and reliability of the collected data.
3. Quality of Service Features: These features reflect the capability of a communication system to reliably and efficiently transmit data.
 - ping: Time for a packet to go to server and back, measured during the test.
 - datarate: Measured throughput during the test (rate of transfer of Data)
 - jitter: Variation in packet delay (latency) over time.

4. Network Data: Network data is a representation of the key characteristics of a communication system, which includes features such as quality of signal, operational frequencies and radio resource indicators. These parameters are necessary to evaluate the performance and efficiency of the system. In the context of this dataset, primary cell and secondary cell are denoted by PCell and SCell correspondingly.

- Signal Data: These features are used to represent the quality of signal.
 - RSRP: Reference Signal Received Power, measures of signal strength in dBm.
 - RSRQ: Reference Signal Received Quality, measures signal quality and accounts for interference/load effects.
 - RSSI: Received Signal Strength Indicator; measure for total received power including signal, interference, and noise.
 - SNR: Signal-to-Noise Ratio for the serving cell; higher values indicate cleaner signal.
- Radio Resource Control (RRC): These features represent key parameters of a cellular connection, including bandwidth, operating frequencies, and cell identity information.
 - Downlinkg/Uplink frequency: Frequency(MHz) used by network or vehicle to send data in Downlink or Uplink direction
 - Downlink/Uplink Bandwidth: Spectrum width allocated for Downlink/Uplink
 - Number of Resouce Blocks (RBs): Count of scheduled RBs allocated in the uplink/downlink directions, which represent small units of radio capacity used for data transmission.
 - Number of Resources Blocks in Modulation and Coding Scheme(MCS) 0-31: This indicates the number of RBs used by serving cell to send data in downlink using a particular Modulation and Coding Scheme.
 - Cell ID/Cell Identity: Global/Local identifier (physical cell identity component, depending on logger) for the serving cell.
 - TAC: Tracking Area Code of the region.
 - MCC/MNC/MNC-digit: Mobile Country Code(MCC) or Mobile Netwrok Code (MNC) of Serving Cell

5. Temporal Features and Identifiers: These features are not directly related to the communication system performance but are important for organising and interpreting the data in a relevant context.

- Temporal data: timestamps/ts-gps indicates the duration of measurement and timestamp associated with GPS.
- Identifiers: Each measurement equipment is recognised by a unique device ID, and the driving mode utilised during data collection, e.g., platooning or 2×2 mode.
- Metadata: The dataset also includes metadata such as Scenerio which indicates test context(uplink, downlink, datarate, latency) as well as direction of travel in order to distinguish the test route followed by the vehicle.

Following the description of the dataset and its feature groups, the subsequent chapters present the analysis of the Berlin V2X dataset[54] and discuss different machine learning techniques used for data imputation and synthetic data generation.

4 Data Imputation on Berlin V2X Dataset

This chapter presents the data preparation and implementation of data imputation model on the Berlin V2X cellular dataset[65]. Section 3.2 describes the dataset comprising 159 features, which include GPS position, traffic state, weather data, QoS measurements, and physical-layer features from PCell and SCell of each vehicle modem[65]. Many of these features contain missing values due to missing secondary carriers, measurement gaps, or complete recording failures. To handle these missing values and reconstruct incomplete data sequences, the implementation is divided into two phases: data analysis and data imputation using a GRU-based model.

4.1 Data Analysis

The data analysis phase forms the foundation for all subsequent preprocessing and modeling decisions. It consists of complete characterization of the raw dataset in three dimensions: the overall structure and data types, the distribution and pattern of missing values in feature groups, and the correlations between features that drive both the removal of redundant information and the grouping of features into physically meaningful categories.

The cellular dataset has a large number of features, as explained in the section 3.2, however it is important to determine and select the most relevant features for the objective of this work. The selection and separation of these features depend on numerous parameters such the percentage of missing data, correlation among features and type of data represented by each feature. Thus, by taking these elements into consideration, only relevant and reliable features are maintained for further analysis and model development.

Based on the above-mentioned criteria, the features can be categorised into high-priority and low-priority groups. High-priority features are those that are highly relevant to the target variable. On the other hand, low-priority features are the ones that contribute minimally to the learning process, have a significant amount of missing data, or are highly correlated. This categorisation reduces computational complexity, simplifies feature selection and increases the efficacy of the machine learning models.

The features with high priority include GPS data, QoS-related features, network data and temporal features. GPS and temporal features are important to capture the contextual information about the geographical position of the vehicle and the timestamp for corresponding measurement. These features are essential to correlate and map measurements across different feature groups and therefore serve as key reference variables in the dataset.

The network data feature category contains measurements associated with PCell and SCell. However, the features related to the SCell have a missing data percentage of about 80%, which significantly limits their reliability and usability for machine learning tasks. These Features are considered low-priority features since they possess high sparsity, which indicates these features are unlikely to provide important information during model training. Furthermore, certain network identity features, such as Cell Identity, Area Code, and Country Code contain nearly constant or low-variance values throughout the dataset. These features possess limited distinctive information and can adversely impact model performance by introducing redundancy or bias.

Therefore, continuous network parameters, signal data, and RRC parameters, together with QoS features, are categorised as high-priority features due to their strong relevance to network behaviour and communication performance. In contrast, feature groups such as environmental and weather-related data originate from external data sources and are expected to have limited influence on the prediction of network-related parameters within the V2X dataset. Therefore, these features are classified as low-priority features for the scope of the thesis.

4.1.1 Types of Missing Data

After loading the dataset and resetting the index, an initial overview was performed with descriptive statistics and an inspection of all available columns. The dataset was found to contain a mixture of numeric features, including continuous measurements and integer-valued radio parameters, categorical features such as device identifier, operator, direction, QoS mode, scenario, and drive mode, as well as datetime features. The 159 raw columns were then organized into nine semantic groups based on their measurement source and functional meaning. Table 4.1 summarizes each group together with its feature count and average missing-value percentage.

The overall missingness across the raw dataset is approximately 26%. However, missing data in the cellular dataset is neither uniform nor random across all features. Therefore, characterizing the missingness pattern of each feature group is essential before selecting an appropriate imputation strategy. Based on the missing data mechanisms described in section 2.5, the features are categorized according to their missingness characteristics.

The missing values in the GPS-related features, including Latitude, Longitude, Altitude, speed_kmh, and COG, show characteristics of Missing Completely at Random, as their missingness appears to be caused by occasional measurement or recording gaps rather than by the values of the variables themselves. In this case, the probability of a missing value is assumed to be independent of both observed and unobserved variables.

Tab. 4.1: Feature groups and average missingness in the Berlin V2X cellular dataset

Feature Group	#Feature	Avg. Missing %	Examples
Temporal	2	0%	timestamp, ts_gps
Identifiers	8	0–5%	device, operator, scenario, direction
GPS / Mobility	7	~1–2%	Latitude, Longitude, speed_kmh, COG
Weather	11	~1–2%	temperature, humidity, wind speed
Traffic	3	~1–2%	Traffic Jam Factor, Traffic Distance
Network QoS	3	~10–30%	datarate, ping_ms, jitter
PCell Signal	21	~5–15%	RSRP, RSRQ, RSSI, SNR
PCell DL / UL	42	~5–80%	MCS, RBs, TB Size, Tx Power
SCell Features	53	~70–100%	Secondary cell signal and radio data

In contrast, the SCell feature group can be categorized as Missing at Random, since the availability of SCell measurements depends on observable network conditions. When there is no active cell aggregation, the connection is represented only by PCell indicators, leaving the corresponding SCell features empty. This relationship is supported by the quantitative analysis, which confirms that SCell measurements are present only when PCell is active, with very few exceptions. Similarly, QoS characteristics (ping_ms, datarate, jitter) exhibit consistent missingness that correlates with measurement direction and scenario.

Missing Not at Random describes a situation in which the likelihood of a value being missing is dependent on the value that is unobserved. In the C-V2X dataset, this situation may occur during severe network degradation, where the absence of a measurement is directly related to the poor channel condition that the missing value would have represented. The Figure 4.1 is an exemplary illustration of the missing data pattern for some of the features of the dataset. The percentage of missing data observed during the three-day measurement campaign for Ping, GPS features, and SCell features is indicated in the top-left figure. The top-right figure shows the hourly distribution of missing data for these features throughout the day. In addition, the bottom left figure shows the variation in the number of collected measurements over different hours, and the bottom right provides a device-wise analysis of Ping missingness across different measurement dates.

4.1.2 Data Cleaning

Before applying machine learning or deep learning models, the raw dataset must be cleaned by removing unwanted features, correcting data types, and excluding records that provide little or

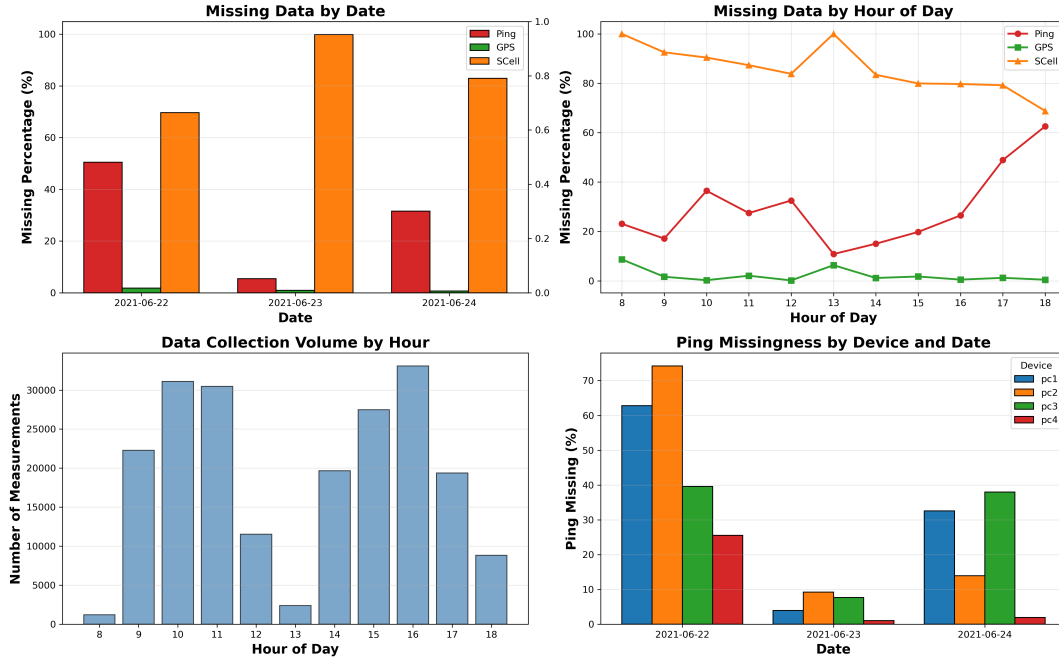


Fig. 4.1: Missing Data in Cellular Dataset

no useful information for model training. In this work, the data cleaning process is divided into three main categories:

- **Row removal:** Approximately 2492 rows, corresponding to about 1.2% of the dataset, were removed because GPS, weather, traffic, and area-related information were simultaneously absent. These rows are mostly initialisation periods at the beginning of measurement sessions, when contextual data were not actually recorded. Since the imputation and data generation models rely on GPS-based seed information, rows without positional context are not useful for model development. Therefore, their removal is considered reasonable and does not significantly bias the remaining dataset.
- **Type correction:** Four bandwidth columns, corresponding to PCell and SCell DL/UL bandwidth values in MHz, were stored in text format in the raw dataset. Therefore, these columns were converted to numeric values for further use. Any non-numeric entries generated during this conversion were treated as missing values and handled later by the downstream imputation pipeline.
- **Feature Removal:**
 - **Completely missing features:** Three SCell uplink features, namely *SCell_Uplink_Num_RBs*, *SCell_Uplink_TB_Size*, and *SCell_Uplink_Tx_Power*,

contained no observed values in the dataset. Since features with 100% missingness provide no learnable information to the model, they were removed.

- Quasi-constant features: Seven features were either completely constant or had more than 98% identical values throughout the dataset. These included visibility, PCell/SCell MCC and MNC codes, and area codes. Since these features mainly represent network operator identifiers or information that does not vary meaningfully within the measurement campaign, they do not provide discriminative information for machine learning or deep learning models. Therefore, they were removed to reduce unnecessary input dimensionality.
- Redundant correlated features: A pairwise correlation analysis was performed on the numeric columns to identify highly correlated features. The feature `apparentTemperature` was very strongly correlated with `temperature`. Therefore, the feature `apparentTemperature` was dropped, and the `temperature` was maintained as the main feature.
- Low-information features: The weather-related features `cloudCover`, `pressure`, `dewPoint`, and `uvIndex` were removed because they showed limited variation during the three-day measurement campaign. Removing them helps simplify the feature space while preserving the variables that are more relevant for V2X communication analysis.

4.1.3 Feature Engineering

After removing uninformative, redundant, and missing features, the retained dataset undergoes a comprehensive feature engineering stage to improve data representation, preserve relevant temporal and network-related information, and prepare the dataset for machine learning and deep learning model development.

- MCS Feature Aggregation: The original dataset contains 32 individual columns `PCell_Download_RBs_MCS_0` through `PCell_Download_RBs_MCS_31` representing the number of resource blocks allocated at each MCS index. These 32 columns are highly correlated within each modulation order group and individually sparse. They are aggregated into three summary columns corresponding to the three modulation schemes:

- `PCell_DL_RBs_MCS_Low`: Total number of RBs of MCS indices 0–9 (QPSK)
- `PCell_DL_RBs_MCS_Mid`: Total number of RBs of MCS indices 10–16 (16-QAM)
- `PCell_DL_RBs_MCS_High`: Total number of RBs of MCS indices 17–31 (64-QAM/256-QAM)

This aggregation reduces 32 sparse columns to 3 dense and physically interpretable features while preserving the essential distribution of downlink modulation usage. The 32 original MCS columns are then dropped.

- Log transformation: Many numerical features in the dataset were highly right-skewed, with values varying over several orders of magnitude. To reduce the effect of extreme values and improve the suitability of these features for model training, a logarithmic transformation was applied before scaling. Thirteen features were transformed using the $\log_{1p}$ function, which is defined as $\log(1 + x)$.

The transformed features include *datarate*, *target_datarate*, *PCell_Downlink_TB_Size*, *PCell_Uplink_TB_Size*, *PCell_DL_RBs_MCS_Low*, *PCell_DL_RBs_MCS_Mid*, *PCell_DL_RBs_MCS_High*, *PCell_Downlink_Num_RBs*, *PCell_Uplink_Num_RBs*, *ping_ms*, *Pos in Ref Round*, *Traffic Distance*, and *PCell_Uplink_Tx_Power_(dBm)*.

This transformation is particularly useful for variables such as *datarate*, which can range from near-zero values to several million bits per second. By compressing the scale of large values, the log transformation reduces skewness and produces distributions that are more suitable for gradient-based machine learning and deep learning models.

- Missing Value Indicators: For features in the informative missingness range (5–50% missing), binary indicator columns with the suffix *_missing* are created. These indicators encode whether each observation was originally observed or absent. The GRU-D imputation model[66] can use these indicators as an explicit mask signal in addition to the temporal decay mechanism.
- Categorical Encoding: Six low-cardinality categorical features were one-hot encoded: device (pc1–pc4), area, scenario, drive_mode, direction (DL/UL), and measured_qos (datarate/delay). One-hot encoding was applied without drop_first to avoid imposing an implicit reference category, which is appropriate for deep learning models that learn their own weighting of each level. High-cardinality string identifiers (such as cell identity text labels) were dropped, as they produce impractically sparse representations. The operator column (values 1 and 2) was binary-encoded to 0 and 1.
- Temporal Features: Hour of day (0–23) and day of week (0 = Monday, 6 = Sunday) were extracted from the timestamp column. Such temporal information is relevant because network performance can vary throughout the day due to changes in user demand, traffic

density, and network load.

4.2 Data Imputation Methodology

The data imputation phase aims to recover missing values in the preprocessed V2X dataset using a deep learning model that explicitly accounts for the temporal structure and informative patterns of missingness identified during the analysis phase. This section describes the baseline methods used for comparison, the chosen GRU-D architecture, the implementation details, and the feature-wise evaluation of imputation quality.

4.2.1 Data Preparation for Data Imputation

Upon completion of the feature engineering stage, the engineered dataset is prepared for the imputation pipeline by several final preprocessing steps. These include temporal train-test split, outlier clipping, and the creation of artificial masks.

- **Temporal split:** The dataset is split into training, validation and test sets in chronological order with a ratio of 70%, 15% and 15% respectively. To this end, all rows are first sorted by the timestamp and then split at fixed temporal boundaries. This approach ensures that the model is evaluated only on future time periods that were not observed during training. Such a split better reflects a realistic deployment scenario, where historical measurements are used to impute missing values or generate future sequences.

Tab. 4.2: Temporal data split

Split	Proportion	Purpose
Training	70% (first in time)	Model parameter learning and scaler fitting
Validation	15% (middle)	Hyperparameter tuning and early stopping
Test	15% (last in time)	Final performance metrics (RMSE, MAE)

- **Feature Scaling and Outlier clipping:** Before scaling, each numerical feature is clipped using the 0.5th and 99.5th percentiles computed from the training data. This outlier clipping step reduces the influence of extreme values that could otherwise distort the mean and standard deviation during standardization. After clipping, StandardScaler is applied to standardise the features. StandardScaler was selected over MinMaxScaler because the validation and test sets are chronologically later than the training set and may contain

values outside the range observed during training. In such cases, MinMaxScaler can map values outside the expected $[0,1]$ interval, whereas StandardScaler is not bounded and is therefore more suitable for handling out-of-distribution values.

- **Artificial masking for evaluation:** In the context of missing data imputation, model evaluation necessitates access to the true values of the imputed entries. However, naturally missing values within the dataset remain unobserved and thus cannot be utilized directly for quantitative assessment. Following the standard approach in imputation benchmarking[66], an artificial masking procedure was applied to the validation and test sets. In this procedure, 15% of the observed values in each feature were randomly selected, stored as ground truth, and temporarily replaced with missing values. The imputation model reconstructs the masked positions, and the predicted values are tested against the ground truth using Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). The training set is not masked, so the machine learning model can learn as much as possible from the observed data.

After the preparation processes, the dataset is ready for the imputation stage. In this stage, different imputation approaches are employed to estimate missing values and reconstruct incomplete feature entries

4.2.2 Data Imputation Models

As mentioned in section 2.5.1, kNN is used as a baseline method for the purpose of data imputation. In this work, k-nearest neighbours is implemented with $k=5$, imputing each missing value by computing the mean of five most similar observed rows using the Euclidean distance over the non-missing features[29]. While kNN captures cross-feature relationships, it does not model temporal dependencies and treats each row independently. To address this limitation, a neural network-based approach, DAE, is introduced as mentioned in section 2.5.1.

In this work, the DAE is implemented as a fully connected, symmetric encoder-decoder architecture. It uses 94 input features, consisting of 35 continuous target features and 59 contextual features, including missingness indicators, temporal features, and one-hot encoded categorical variables. The output layer reconstructs only the 35 continuous features selected for imputation. Table 4.3 summarises the full architecture and training configuration.

Tab. 4.3: DAE architecture and training configuration

Parameter	Value
Input dimension	94 (35 continuous + 59 context features)
Output dimension	35 continuous features
Architecture	$94 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 35$
Activation / Dropout	ReLU + Dropout(0.20) on all hidden layers
Loss / Optimiser	Weighted masked MSE Adam, lr = 0.001
Early stopping	Patience = 10 epochs
Best val loss	0.2133 (epoch 39)
Training time	849.4s (≈ 14.2 min)

The model is trained using a weighted masked mean squared error (MSE) loss, calculated only over the observed values. The feature weights are assigned inversely proportional to feature variance to prevent high-variance features from dominating the gradients during optimization. The training was carried out for 40 epochs, with the best validation loss of 0.2133 achieved at epoch 39.

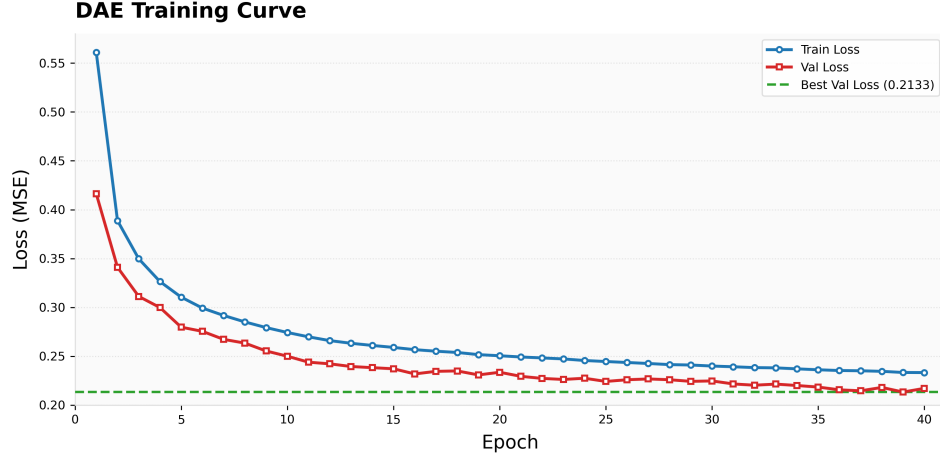


Fig. 4.2: DAE Training Loss

Although the DAE improves over simpler baseline methods by learning nonlinear cross-feature relationships, it has an important architectural limitation: it processes each row independently and therefore does not explicitly model the temporal ordering of observations. This limits its suitability to sequential V2X measurements, where the evolution of network and mobility features over time is crucial for accurate imputation.

RNNs address this limitation by maintaining a hidden state over time, enabling the model to make predictions based on previously observed data [67]. However, the conventional RNNs and LSTMs do not have an efficient mechanism to capture the meaningful missingness patterns that are observed in real-world data, as they often treat missing information as zeros or require complete inputs[66]. GRU-D was chosen precisely because it extends the traditional GRU architecture to meet both difficulties at the same time: collecting temporal dependencies of sequential data and explicitly modelling missing pattern and time gaps as important inputs rather than considering them as noise[66]. The schematic diagram of a GRU-D architecture can be illustrated from Figure 4.3

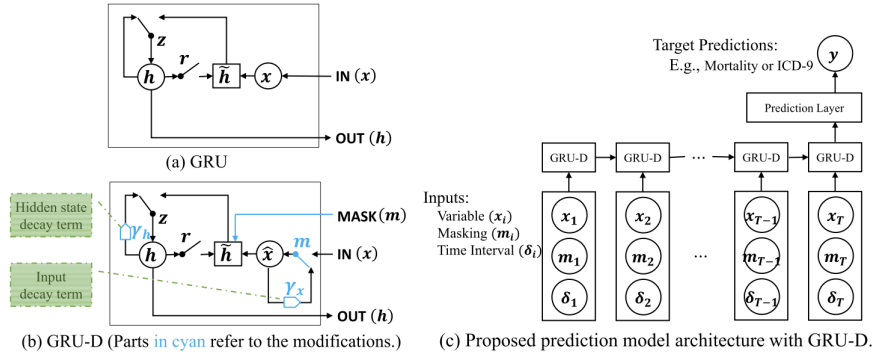


Fig. 4.3: GRU-D model architecture[42]

The GRU-D architecture used for data imputation has total input dimension of 51 features (35 continuous + 16 categorical encoded features), with 9 static features (device identity, operator, direction, QoS mode, temporal indicators) concatenated at each timestep. The model was trained using weighted masked MSE loss, and early stopping with patience = 15 epochs. Training stopped at epoch 37 with best validation loss of 0.0993.

The training curve illustrated in Figure 4.4 for GRU-D achieved a best validation loss of 0.0993, compared to 0.2133 for the DAE. This reflects the ability of GRU-D to exploit temporal structure and informative missingness patterns that the row-independent DAE cannot access.

Tab. 4.4: GRU-D architecture and training configuration

Parameter	Value
Input dimension	51 (35 continuous + 16 categorical + 9 static)
Output dimension	35 continuous features
Architecture	GRU hidden=128, seq_len=20 $\rightarrow$ Linear(128 $\rightarrow$ 35)
Activation / Dropout	ReLU + Dropout(0.30)
Loss / Optimiser	Weighted masked MSE Adam, lr = 0.001, wd = 1e-5
Early stopping	Patience = 15 epochs
Best val loss	0.0993 (epoch 22)
Training time	4,019.5s ($\approx$ 66.9 min)

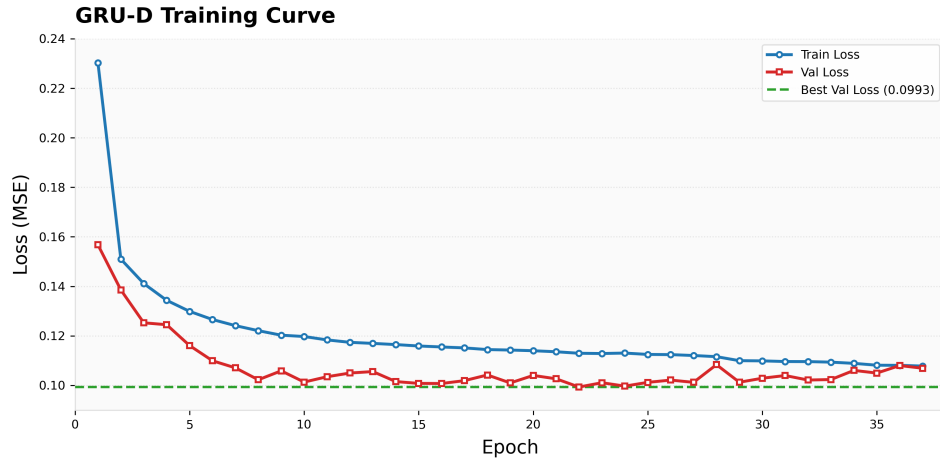


Fig. 4.4: GRU-D Training Loss

4.2.3 Evaluation of Data Imputation Model

All models were evaluated on the held-out test set using the artificial masking procedure from section 4.2.1. Overall RMSE and MAE are reported across all evaluated features and masked positions for each model.

After training, the model is evaluated on the held-out test set, which comprises the final 15% of the timeline and was not used for training or hyperparameter tuning. To assess imputation performance, 15% of the observed test values are artificially masked, while their original values are retained as ground truth. The trained model then predicts these masked values and the predictions are compared with the ground truth using RMSE and MAE. This evaluation combines temporal holdout with artificial masking, ensuring that the model is tested on unseen time periods and on values intentionally hidden during evaluation. The Table 4.5 presents the resulting

feature-wise imputation performance.

Tab. 4.5: Feature-wise imputation performance of the GRU-D model

Feature	RMSE	MAE
Latitude	0.0022	0.0017
jitter	0.0027	0.0016
Longitude	0.0095	0.0077
precipIntensity	0.0548	0.0504
Pos in Ref Round	0.2512	0.1785
speed_kmh	0.5877	0.4371
Traffic Distance	0.5884	0.4519
PCell_Uplink_Tx_Power_(dBm)	0.6179	0.4699
ping_ms	0.6857	0.5455
temperature	0.7036	0.6144
PCell_Band_Indicator	0.8606	0.5552
PCell_DL_bandwidth_MHz	0.8675	0.5536
PCell_UL_bandwidth_MHz	0.8735	0.5758
target_datarate	0.9899	0.8276
datarate	1.0029	0.8823
PCell_Uplink_TB_Size	1.2885	1.0126
PCell_Uplink_Num_RBs	1.3527	1.1354
Traffic Jam Factor	1.3617	0.8680
PCell_DL_RBs_MCS_Low	1.4443	1.0685
PCell_RSRQ_max	1.6481	1.2720
PCell_Downlink_Num_RBs	1.8372	1.5085
PCell_Downlink_TB_Size	1.9800	1.6274
PCell_DL_RBs_MCS_High	2.1477	1.8099
PCell_DL_RBs_MCS_Mid	2.4868	2.0120
PCell_SNR_1	2.7627	2.1549
PCell_Downlink_Average_MCS	2.9870	2.2983
PCell_SNR_2	3.0491	2.3529
PCell_RSSI_max	3.4482	2.6011
measurement	4.4972	4.2087
Altitude	4.5871	3.6334
PCell_RSRP_max	4.9443	3.9470
COG	29.7075	22.0720

4.2.4 Data Imputation Results

The cellular dataset contained approximately 26% missing values across the selected 35 continuous features, corresponding to more than 158,000 masked evaluation points in the test set. To address these missing values, three imputation approaches were evaluated, each representing a different level of modeling capability. The performance of employed imputation methods can be observed in the Table 4.6.

Tab. 4.6: Performance comparison of imputation models

Model	Test RMSE	Test MAE	Eval. Points
kNN ($k = 5$)	14,771.35	41.116	–
DAE	10.71	2.99	158,362
GRU-D	5.45	1.80	158,265

kNN showed the weakest performance, mainly because it is sensitive to differences in feature scales and does not explicitly model temporal dependencies. The DAE substantially improved over kNN by learning cross-feature relationships through its bottleneck architecture, while GRU-D achieved the best overall performance, with a test RMSE of 5.45 and a test MAE of 1.80. The improvement is mainly attributed to ability of GRU-D to model informative missingness and temporal decay, which is particularly beneficial for signal-quality features such as RSRQ, SNR, and RSSI, where temporal structure is highly relevant.

The improvement is most pronounced for high-priority network signal-quality features such as RSRQ, SNR, and RSSI. For example, GRU-D reduces the RMSE for RSRQ from 2.47 with the DAE to 1.65, for SNR from 4.16-4.51 to 2.76-3.05, and for RSSI from 10.5 to 3.45. These features are essential for effective network planning and QoS analysis.

The stronger performance of GRU-D can be explained by its ability to model temporal dependencies. The quality of the signal at a specific timestamp depends on the measurements in the preceding seconds and GRU-D models this behaviour through its sequential hidden state and temporal decay mechanisms. In contrast, kNN and DAE process samples independently, such that neither explicitly uses the temporal context before a missing gap.

After evaluation, the trained GRU-D model was applied to the complete dataset to fill the remaining missing values in the 35 continuous features. Imputation was performed separately for each data split, resulting in 231,042 values filled in the training set, 11,074 in the validation set, and 20,138 in the test set. In total, 262,254 missing values were imputed across the full dataset.

A final verification confirmed that no NaN values remained in any of the selected continuous features across all splits. The resulting complete dataset containing 204,942 rows and 72 columns were saved and used as the input for the subsequent synthetic data generation pipeline.

Tab. 4.7: Missingness statistics before and after GRU-D imputation

Feature	Missing Before	% Missing	Values Imputed (GRU-D)
ping_ms	57,707	28.16	57,707
datarate	15,679	7.65	15,679
jitter	15,679	7.65	15,679
Pos in Ref Round	14,159	6.91	14,159
PCell_Band_Indicator	12,607	6.15	12,607
PCell_UL_bandwidth_MHz	12,607	6.15	12,607
PCell_DL_bandwidth_MHz	12,607	6.15	12,607
PCell_Uplink_Num_RBs	12,210	5.96	12,210
PCell_Uplink_TB_Size	12,210	5.96	12,210
PCell_Uplink_Tx_Power (dBm)	12,210	5.96	12,210
PCell_DL_Average_MCS	10,506	5.13	10,506
PCell_DL_TB_Size	10,506	5.13	10,506
PCell_DL_Num_RBs	10,506	5.13	10,506
PCell_SNR_2	10,026	4.89	10,026
PCell_SNR_1	10,026	4.89	10,026
PCell_RSSI_max	10,026	4.89	10,026
PCell_RSRQ_max	10,026	4.89	10,026
PCell_RSRP_max	10,026	4.89	10,026
Traffic Jam Factor	2,878	1.40	2,878
Altitude	91	0.04	91
Total Missing Values	262,254	–	262,254

5 Data Generation for Berlin V2X Dataset

5.1 The concept of synthetic data generation

During the analysis of the cellular dataset, it was observed that each of the four measurement devices either failed to record data or were temporarily halted during the measurement campaign. This resulted in several gaps in the measurement timeline. These missing timestamp intervals were identified during data analysis and were the primary motivation for developing a synthetic data generation model. This model is intended to reconstruct or generate data when complete measurement data is unavailable. The missing timestamp intervals identified for Device 1 and Device 3 are shown in Table 5.1. A similar pattern of missing timestamps was also observed for Device 2 and Device 4.

Tab. 5.1: Identified timestamp gaps for devices pc1 and pc3 on 22 June 2021

Device	Start Time	End Time	Duration
pc1	11:06:00	11:26:15	1215 s (20.2 min)
pc1	12:23:59	14:33:00	7741 s (129.0 min)
pc1	15:44:59	15:56:30	691 s (11.5 min)
pc1	16:26:04	16:33:46	462 s (7.7 min)
pc1	17:16:59	17:24:58	479 s (8.0 min)
pc3	11:06:00	11:26:15	1215 s (20.2 min)
pc3	12:23:59	14:33:00	7741 s (129.0 min)
pc3	15:24:20	15:25:38	78 s (1.3 min)
pc3	15:44:59	15:56:30	691 s (11.5 min)
pc3	17:16:59	17:24:58	479 s (8.0 min)

The generation task begins with a seed window, which is a brief series of real measurements. Using this initial input, the model forecasts feature values for the next timestep. To generate a sequence of desired length, the model takes the predicted timestep as input and this process is repeated iteratively. Recurrent models (such as LSTMs and GRUs) are often used for sequential data production tasks as both of these methods maintain temporal information between timesteps, therefore they can be utilized for data generation purposes.[68]

For the cellular dataset, it is essential to maintain statistical realism. A conventional regression model typically produces sequences that are overly smooth and regular[69], as it predicts mean values rather than capturing rapid fluctuations and variability observed in the actual signal

data. The generated sequences should not only replicate the average behavior of features such as RSRP, datarate, and ping, but also preserve their inherent variability and underlying distribution characteristics.

5.2 Data Generation using GANs

The simplest approach to autoregressive sequence generation is to use a LSTM network that takes all available inputs and predicts all 38 network features at the next timestep simultaneously. This can be used later as a baseline model before adding more complex models, such as GANs, and to better understand the relationships among different features.[68]

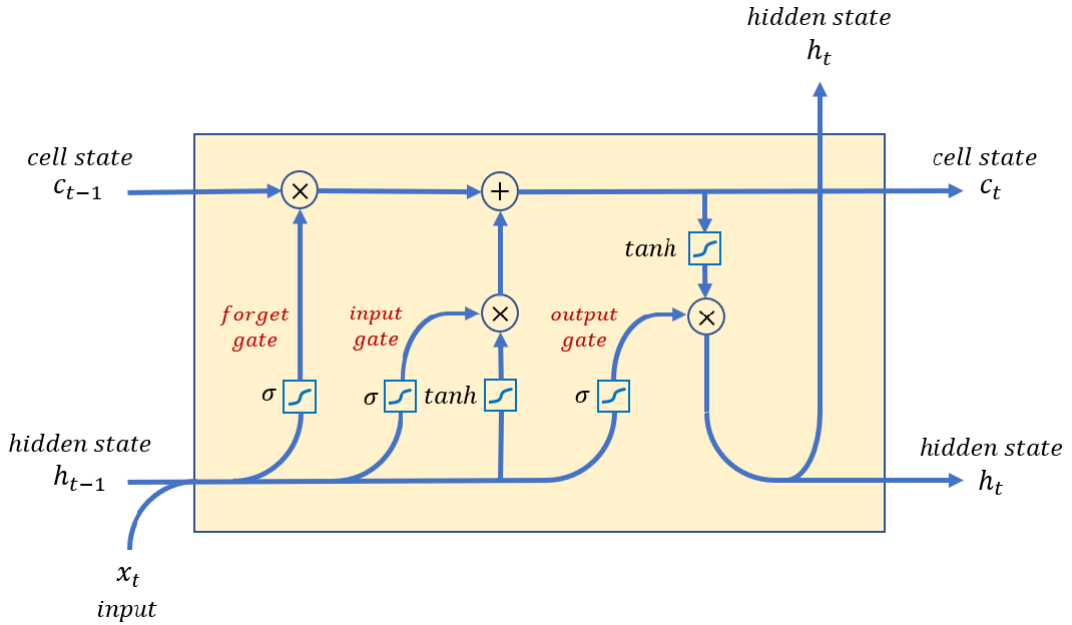


Fig. 5.1: Schematic diagram of LSTM architecture[70]

The model receives two types of inputs at each timestep. The first group consists of base context features including hour, day, device identity, direction, QoS mode, and operator. These features provide fixed conditioning information that defines the measurement setting. Without these context features, the model would not be able to distinguish between different devices, operators, directions, or time-of-day conditions.

The second group consists of the 38 target features, which include signal-quality, radio-resource scheduling, and QoS-related measurements. These target features are used autoregressively during generation. In this process, the model predicts target feature values at timestep t and uses these predictions as input for timestep $t+1$. By repeating this process, the recurrent model

generates a continuous synthetic sequence while preserving temporal dependencies from one timestep to the next. The LSTM model generates all 38 target network features simultaneously from a single shared hidden state. The training parameters used for the LSTM model are illustrated in Table 5.2.

Tab. 5.2: LSTM architecture and training configuration

Parameter	Value
Input dimension	48 (10 base context + 38 target features)
Output dimension	38 target features
Architecture	LSTM (hidden=256, 2 layers, dropout=0.20) $\rightarrow$ FC (256 $\rightarrow$ 128 $\rightarrow$ ReLU $\rightarrow$ Dropout(0.20) $\rightarrow$ 38)
Loss / Optimiser	MSE Adam, lr = 0.0003, wd = 1e-4
LR scheduler	ReduceLROnPlateau (patience = 5, factor = 0.5)
Early stopping	Patience = 15 epochs
Best val loss	0.1996 (epoch 13)
Training time	4,095s ($\approx$ 68.3 min)

While this approach is straightforward, it has a fundamental limitation: all output features are predicted at the same level, without explicitly modelling the physical hierarchy between them. But in case of cellular data, feature relationships are not arbitrary. The GPS position affects the serving cell and radio propagation conditions, which in turn influence signal-quality indicators such as RSRP, RSRQ, RSSI, and SNR. These signal conditions then affect radio-resource allocation, achievable datarate, latency, and other QoS-related metrics. Therefore, a single-step prediction of all features may produce statistically plausible values at the individual-feature level, but still generate unrealistic feature combinations when the underlying physical dependencies are not preserved.

To address this limitation, the target features are grouped according to their physical relationships and generated sequentially. In this hierarchical generation process, the output of one feature group becomes part of the conditioning input for the next group. This hierarchical method is compatible with the causal generation framework presented by Cheng et al.[71], which demonstrates that maintaining feature dependencies throughout the generation process produces more realistic and temporally consistent data.

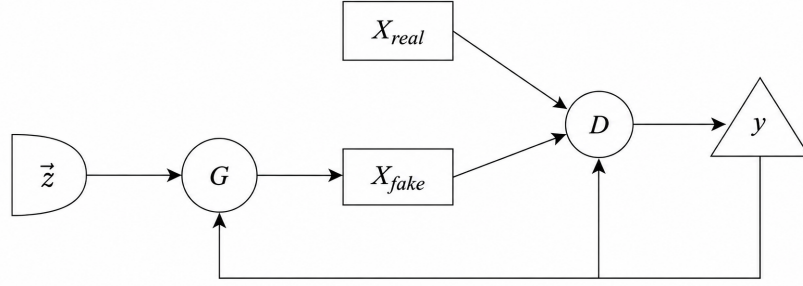


Fig. 5.2: Schematic diagram of GAN architecture[72]

For the GAN model, the 38 target features are divided into five groups that reflect the assumed physical dependency structure of the cellular V2X measurements as illustrated in Table 5.3.

Tab. 5.3: Hierarchical feature grouping for GAN-based synthetic data generation

Level	Name	Target Features
1a	GPS	Latitude, Longitude
1b	Mobility	speed_kmh, sin_COG, cos_COG, Altitude
1c	Environment	precipIntensity, precipProbability, temperature, humidity, windSpeed, Traffic Jam Factor, Traffic Distance
2a	Cell Config	PCell_freq_MHz, DL_frequency, Band_Indicator, DL_bandwidth_MHz, UL_bandwidth_MHz
2b	Signal & QoS	RSRP_max, RSRQ_max, RSSI_max, SNR_1, SNR_2, MCS, RBs, TB Size, datarate, ping, jitter, target_datarate, Pos in Ref Round

Level 1a first generates the GPS coordinates, since vehicle position provides the spatial context for the remaining measurements. Level 1b then generates the mobility-related features, such as speed, heading, and altitude, conditioned on the generated GPS information. Level 1c generates environmental conditions using the position and mobility context. Level 2a generates cell configuration features, which are influenced by the serving cell selected by the vehicle. Finally, Level 2b generates the signal-quality and QoS-related metrics, including RSRP, RSRQ, datarate, ping, and other related features. These final features are conditioned on all preceding levels because they are strongly affected by location, mobility, environmental context, and cell configuration, making them critical for realistic network simulation.

5.2.1 Generation module for GAN

All five levels use a GRU-based generator. GRUs are well suited for autoregressive generation because their gating mechanism helps retain relevant temporal context while filtering out less important past information. Their simpler structure also makes them more parameter-efficient than LSTMs for sequence modelling tasks[73]. For Levels 1a through 2a, a deterministic GRU with a linear output head is used. This is appropriate because these levels mainly contain slowly varying or semi-discrete features, where the objective is to predict the next value of each feature at every timestep. Therefore, supervised training with MSE loss is sufficient for these feature groups. Level 2b contains signal and QoS features such as RSRP, RSRQ, SNR, datarate, and ping present a fundamentally different challenge. These features fluctuate rapidly with changing radio conditions and scheduling decisions, a deterministic generator cannot capture this natural variability, collapsing to smooth, unrealistically regular sequences that do not resemble real measurements. To address this, at level 2b the generator is trained adversarially against a discriminator, forcing it to match the full distributional shape of real signals rather than just their average values[69]. To produce samples that fluctuate as real cellular measurements do, the generator uses a probabilistic output head as described in [74].

The generator uses a two-layer GRU backbone across all five levels, with the hidden dimension adjusted according to the complexity of the target features. For Levels 1a to 2a, a hidden dimension of 128 is used, along with a deterministic output head suitable for slowly varying features. In contrast, Level 2b uses a larger hidden dimension of 256 to model the 19 signal and QoS features. For this level, the probabilistic output head predicts both μ and $\log \sigma^2$ for each feature as described above. Table 5.4 summarizes the generator configuration and training specifications.

Tab. 5.4: GRU-GAN generator architecture and training configuration

Parameter	Value
Architecture (Levels 1a–2a)	GRU (hidden=128, 2 layers, dropout=0.20) → FC (128→64→ReLU→Dropout(0.20)→target dim(19)
Architecture (Level 2b)	GRU (hidden=256, 2 layers, dropout=0.20) → FC (256→128→ReLU→Dropout(0.20)→(19*2) 38)
Parameters (Level 2b)	1,059,091
Loss	MSE + Adversarial loss
Optimiser	AdamW, LR = 0.001, weight decay = 1e-3
LR scheduler	ReduceLROnPlateau (patience = 5)
Best val MSE (Level 2b)	0.0950 (epoch 30)
Training time (all levels)	2,409.1s ($\approx$ 40.2 min)

5.2.2 Discriminator module for GAN

The discriminator evaluates whether a given sequence is real or synthetic. It receives a 20-step context sequence, matching the seed window used during generation, concatenated with the candidate target values. Based on this input, the discriminator outputs a real/fake score indicating how likely the sequence is to come from the real dataset.

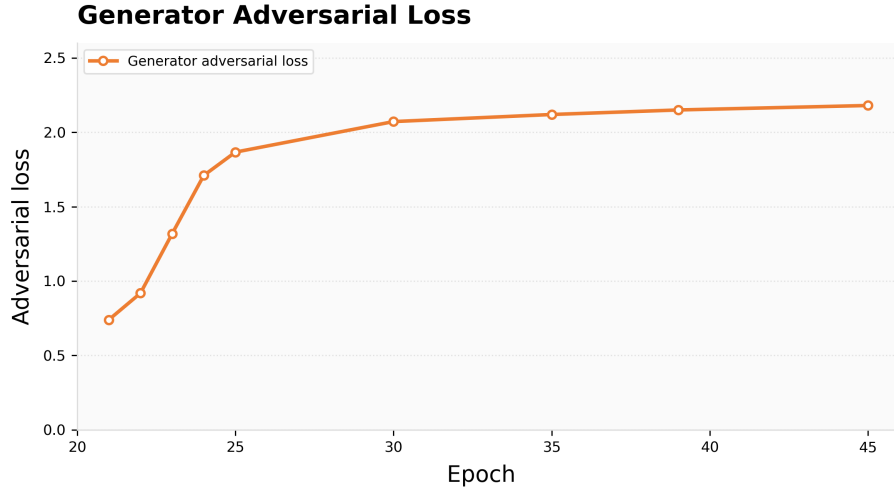
The discriminator is implemented as a single-layer LSTM with a hidden dimension of 64. It processes the input sequence and passes the final hidden representation to a fully connected classification head, which outputs the discriminator score for real or synthetic classification. A lower learning rate is used for the discriminator than for the generator to prevent it from becoming too strong early in training, before the generator has learned the basic signal structure.

Tab. 5.5: Discriminator architecture and training configuration

Parameter	Value
Architecture	LSTM (hidden=64, 1 layer) $\rightarrow$ FC (64 $\rightarrow$ 32 $\rightarrow$ ReLU $\rightarrow$ Dropout(0.30) $\rightarrow$ 1)
Parameters	104,513
Loss	Binary cross-entropy (real / fake)
Optimiser	AdamW, lr = 5×10^{-5} , wd = 1×10^{-4}

Each level is trained independently and sequentially; Level 1a first, then 1b, 1c, 2a, and finally 2b. The outputs of each trained level are frozen before the next level begins training, a standard practice in multi-stage generative pipelines[71]. Levels 1a through 2a are trained converge within 8–28 epochs. At Level 2b, the generator and discriminator are trained simultaneously. Early stopping with patience of 20 epochs is applied, with the best model selected at epoch 30 (validation MSE = 0.0950). Total training time across all five levels is 178.9 mins. The GAN training results confirm that the adversarial learning process functioned as intended. As training progressed, the generator adversarial loss increased from 0.782 to 2.148, while the discriminator loss decreased from 1.338 to 0.748. This shows that the discriminator became more effective at distinguishing real from generated sequences, while the generator adapted in response. As a result, both networks remained in active competition during training. The Figure 5.3 represent the Generator adversarial loss and discriminator loss during the training.

(a)



(b)

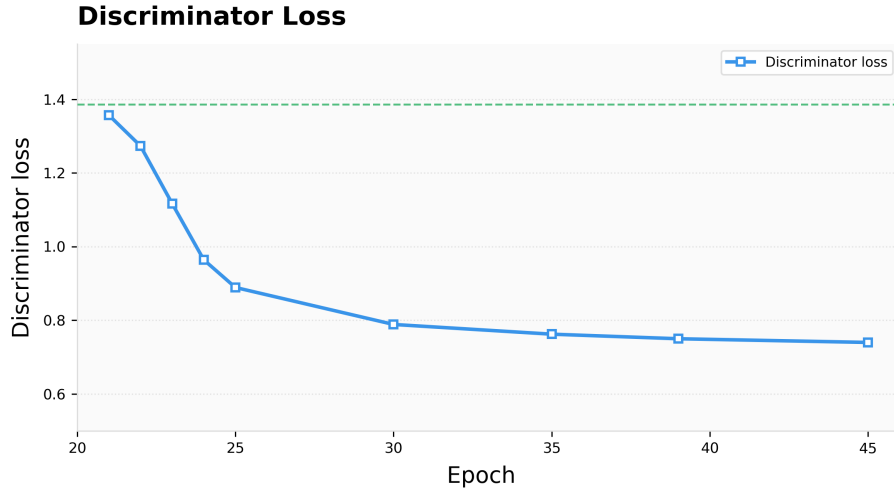


Fig. 5.3: (a) Generator Adversarial Loss and (b) Discriminator Training Loss

5.3 Evaluation of Data Generation Model

After training, both generation models are evaluated on the held-out test set, which corresponds to the final 15% of the timeline and was not used during training or hyperparameter tuning. During evaluation, real ground-truth inputs are provided at each timestep. Model performance is measured using RMSE and MAE, computed after inverse scaling so that the errors are reported in the original feature units. The evaluation is performed across 19,153 test sequences.

Discriminator Evaluation: To further validate the GRU-GAN model, a separate evaluation

discriminator is trained based on the fidelity evaluation framework proposed by Yoon et al.[69]. This method was introduced in TimeGAN[69], provides a quantitative way to assess the realism of generated time-series data. The evaluation discriminator is independent of the discriminator used during GRU-GAN training. It is trained using both real sequences and sequences generated by the trained GRU-GAN, with the objective of classifying each sequence as either real or synthetic. A stronger generator makes classification more challenging since the sequences it generates are more similar to real data.

The evaluation discriminator is a separate LSTM-based classifier with a hidden dimension of 16 and 2,529 trainable parameters. It is trained on 15 signal and 2 QoS features, which were selected because they show the most rapid temporal fluctuations. Both real and generated sequences are passed to the discriminator and the model learns to distinguish between them. Training is performed for 30 epochs using the AdamW optimiser, with early stopping based on validation loss.

Based on the evaluation methodology employed by the TimeGAN authors[69], discriminator performance is assessed using three parameters; fidelity, diversity and usefulness. The degree to which synthetic sequence are identical to genuine data is referred to as fidelity[70]. Diversity assesses how closely distribution of synthetic information resembles that of actual data[70]. The usefulness of the generated data is a measure of how well it supports prediction tasks compared to real data[70].

The discriminative score is defined as:

$$\text{Discriminative Score} = |\text{Accuracy} - 0.5| \quad (5.1)$$

A score of 0.00 indicates that the discriminator cannot distinguish between real and synthetic data, while the worst possible result is a score of 0.50[70]. The initial discriminative score was 0.28 with a discriminator accuracy of 78.26%, indicating that generated sequences were clearly distinguishable from real data. After modifying the feature set and training hyperparameters, the score dropped to 0.0926 with an accuracy of 59.26%. These results suggest that the final generated sequences are much more realistic. Further reduction was not achieved, even with minimal feature configurations.

5.4 Data Generation Results

For synthetic data generation, the GRU-GAN model outperforms the baseline LSTM generation model, particularly for High priority signal and QoS-related features. GRU-GAN achieves

substantially lower RMSE values for key network performance features, including *datarate*, *ping_ms*, *PCell_Uplink_Num_RBs* and *target_datarate*. Improvements are also observed for signal-quality features like RSRQ and RSSI. The Table 5.6 represents that GRU-GAN reduces RMSE by 21–86% for most signal and QoS features, which are the most important features for realistic network simulation. This improvement occurs because the Level-based model generates signal-related features using GPS and mobility context.

Tab. 5.6: Model comparison between LSTM and GRU-GAN models

Feature	RMSE	
	LSTM	GRU-GAN
PCell_RSRQ_max	2.086	1.316
PCell_Downlink_Num_RBs	4.229	0.733
PCell_Uplink_Num_RBs	2.338	0.690
datarate	1.787	0.562
ping_ms	1.302	0.564
target_datarate	2.470	0.339
PCell_RSRP_max	6.588	5.145
PCell_RSSI_max	7.220	4.547
PCell_SNR_1	3.641	3.723
temperature	0.387	1.925
Latitude	0.006	0.011

Featurewise performance of GRU-GAN model can be illustrated in the Table 5.7 below. The results reflect the two main design choices of the GRU-GAN model. First, the target features are organised into five physically ordered groups: GPS, mobility, environment, cell configuration, and signal/QoS. This structure allows signal and QoS features to be generated based on the contextual variables that physically influence them, rather than predicting all features jointly from a single shared hidden state as in the LSTM model. Second, adversarial training combined with a probabilistic output head encourages the generator to produce sequences with realistic step-to-step variability [74]. This helps preserve the natural fluctuations of features such as RSRP, datarate, and ping, instead of producing overly smooth predictions. The Figure 5.4 below represent the performance for the GRU-GAN on high priority features as compared to LSTM model.

Tab. 5.7: Feature-wise evaluation results of the hierarchical GRU-GAN model

Level	Feature	RMSE	MAE
1a GPS	Latitude	0.0109	0.0083
1a GPS	Longitude	0.0537	0.0494
1b Mobility	speed_kmh	0.5370	0.4168
1b Mobility	sin_COG / cos_COG	0.1252 / 0.1239	0.0940 / 0.0972
1b Mobility	Altitude	6.8656	6.6781
1c Env.	temperature	1.9250	1.8957
1c Env.	humidity	0.1103	0.1097
2a Cell	PCell_DL_bandwidth_MHz	0.4861	0.0410
2a Cell	PCell_UL_bandwidth_MHz	0.4861	0.0410
2b Signal	PCell_RSRP_max	5.1447	3.6725
2b Signal	PCell_RSRQ_max	1.3156	0.9817
2b Signal	PCell_RSSI_max	4.5465	3.4597
2b Signal	PCell_SNR_1 / SNR_2	3.7225 / 3.7438	2.9032 / 2.8992
2b Signal	PCell_DL_Average_MCS	3.3280	2.5427
2b Signal	PCell_DL_RBs_MCS Low/Mid/High	1.783 / 2.975 / 1.203	1.360 / 2.028 / 0.734
2b Signal	PCell_UL_Num_RBs	0.6897	0.5154
2b Signal	PCell_UL_TB_Size	0.7869	0.5781
2b Signal	PCell_UL_Tx_Power_(dBm)	0.6481	0.4280
2b QoS	datarate	0.5619	0.4089
2b QoS	jitter	0.3638	0.0242
2b QoS	ping_ms	0.5640	0.4117
2b QoS	target_datarate	0.3388	0.1963

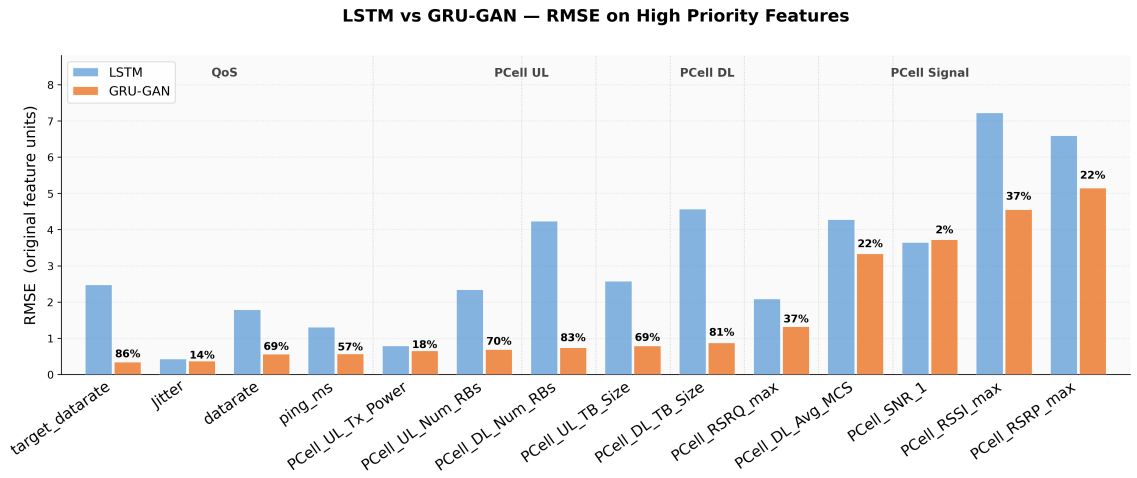


Fig. 5.4: LSTM vs GRU-GAN RMSE for High Priority Features

However, some limitations remain. The LSTM performs better for slowly varying contextual features such as temperature and GPS, where the level based structure provides less advantage. For example, the GRU-GAN obtains an RMSE of 1.93 for temperature and 0.011 for Latitude, both of which are higher than the corresponding LSTM results. Altitude also shows a relatively large RMSE of 6.87, indicating that the model does not improve all feature groups equally.

In addition, the evaluation was limited to single-step prediction with ground-truth inputs. A full autoregressive evaluation, where the model's own predictions are fed back over longer generated sequences, was not completed and is therefore left for future work.

5.5 Representation of Real and Synthetic Data

The Figure 5.5 below represents feature-wise distribution of Real data as compared to Synthetic data for Signal and QoS Level Features. To verify the realism of the synthetic data, the GRU-GAN model was used to generate sequences corresponding to the real timestamp structure. The generated data distribution was then compared with the original measurement data using distribution plots. The overlap between the real and synthetic data distributions indicates that the generated data replicates the statistical behaviour of the real dataset.

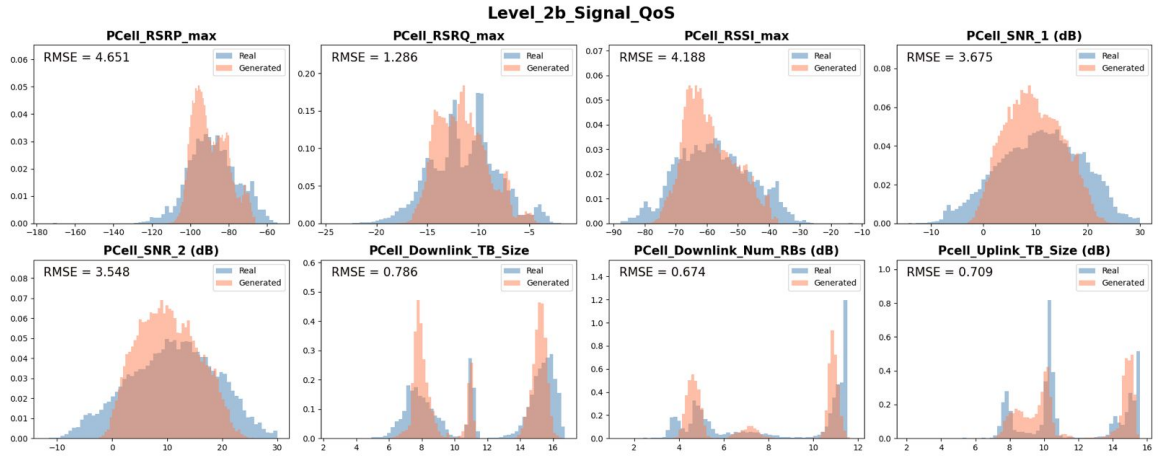


Fig. 5.5: Real vs Generated Feature Distribution

Furthermore, to evaluate the generated data points, the synthetic values were compared against real measurements at 195 sampled points from the same day. These sampled points correspond to data from Device 1 on Day 1. The feature-level results show strong agreement between the generated and real measurements. Correlation values above 0.92 across the selected signal and QoS features confirm that the generated sequences closely track real signal dynamics. Speed achieves the highest correlation of 0.996, reflecting smooth GPS-conditioned mobility

generation. RSRP and SNR also show strong correlations of 0.956 and 0.921, respectively, despite the variability of signal quality in urban driving conditions. Ping achieves a near-perfect correlation of 0.990. The feature-level distributions are shown in Figure 5.6.

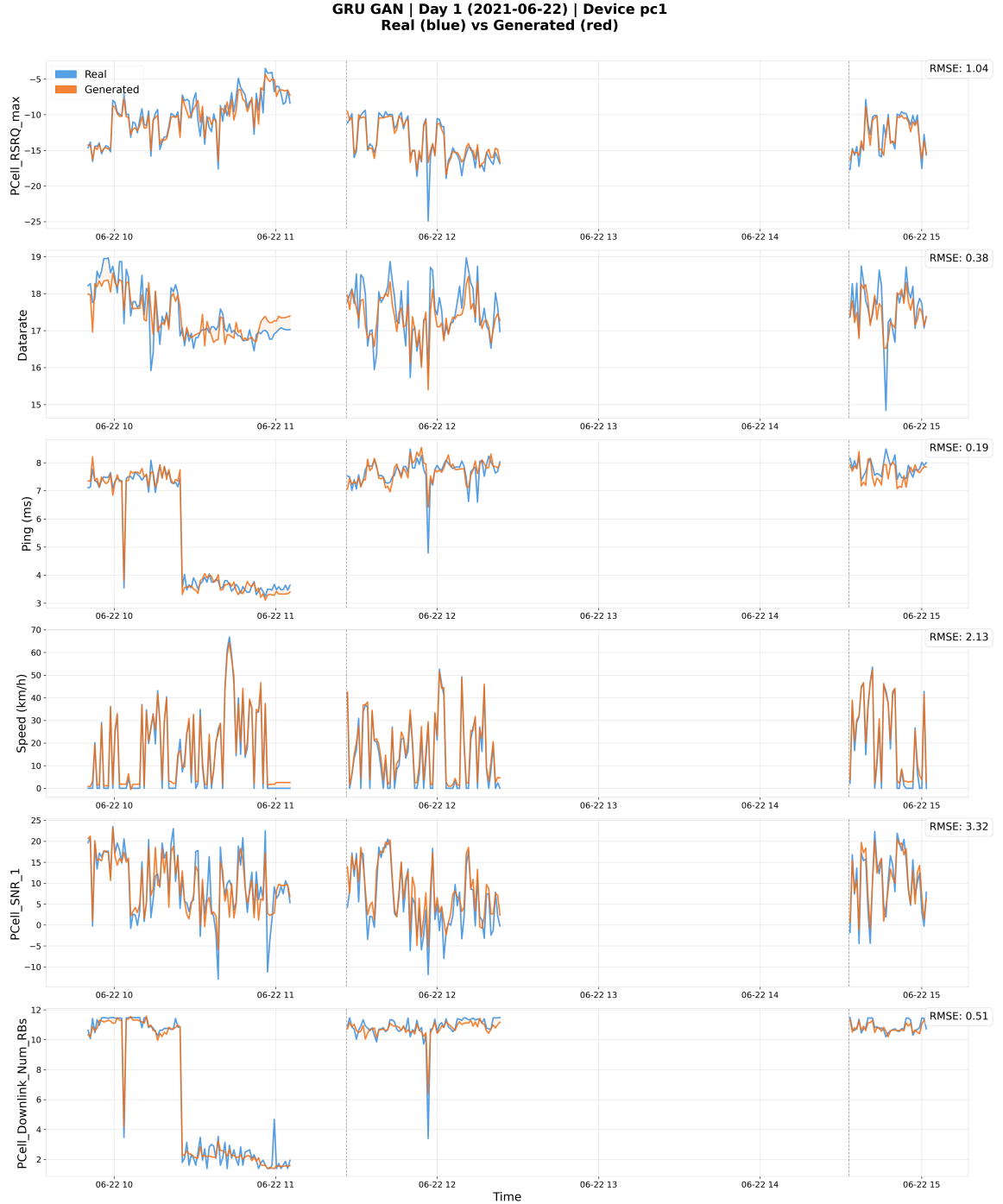


Fig. 5.6: Overlap between the real and synthetic distributions

The original use case of filling missing timestamp gaps was tested using the trained GRU-GAN model. Specifically, the model was applied to reconstruct a 7.7-minute gap (462 seconds) identified for device pc1 on Day 1 of the measurement campaign, 22 June 2021, between 16:26 and 16:34, as shown in Table 5.1. Based on this information, the GRU-GAN generated 461 synthetic timesteps to cover the missing period. This experiment demonstrates how the proposed generation module can be used to reconstruct missing measurement intervals and improve the temporal continuity of the V2X dataset. The Figure 5.7 below can be used to illustrate the filled gaps.

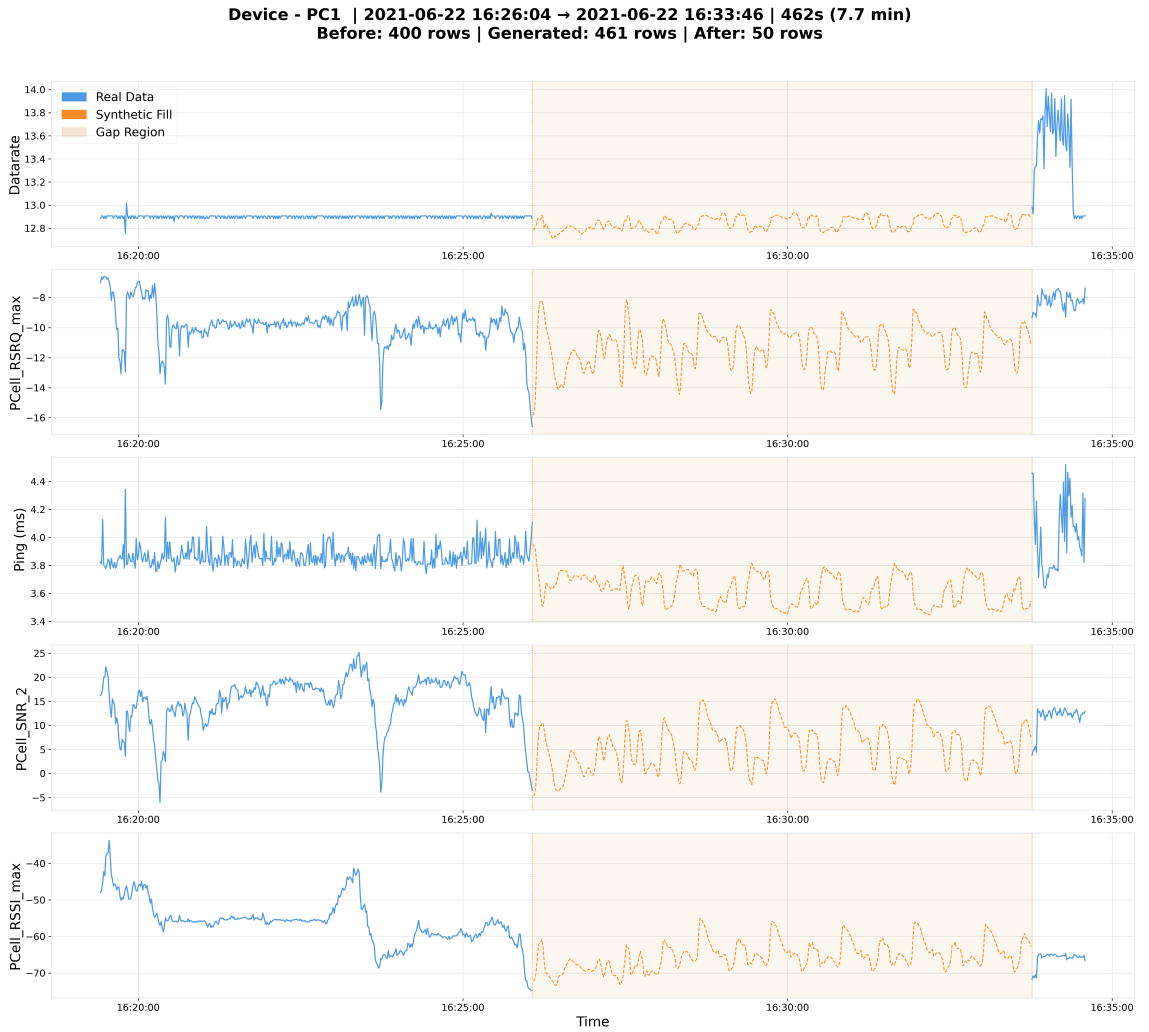
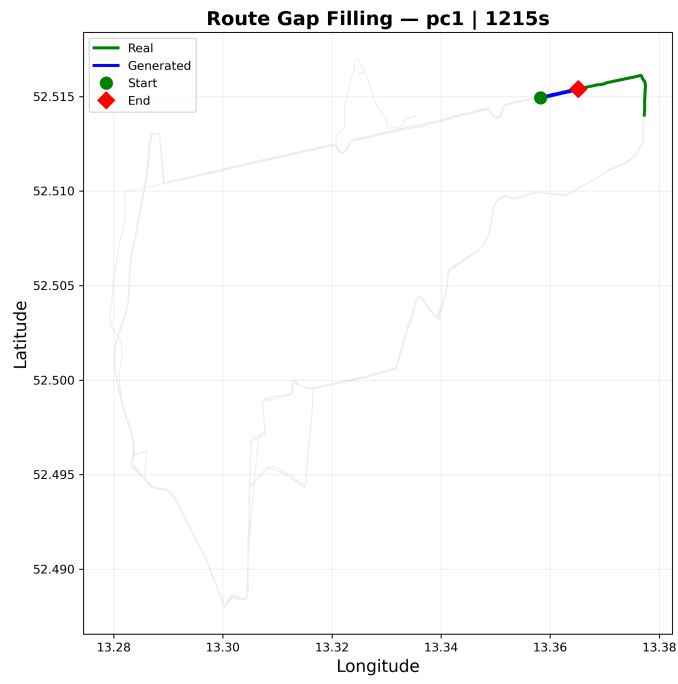


Fig. 5.7: Data gaps filled using Synthetic Data

To further validate the generated GPS data, the model was also used to predict GPS points within missing timestamp intervals for different devices. This evaluation helps verify whether the generated trajectories remain consistent with the expected movement pattern during measurement gaps. The generated routes for Device 1 and Device 2 are shown in Figure 5.8.

(a)



(b)

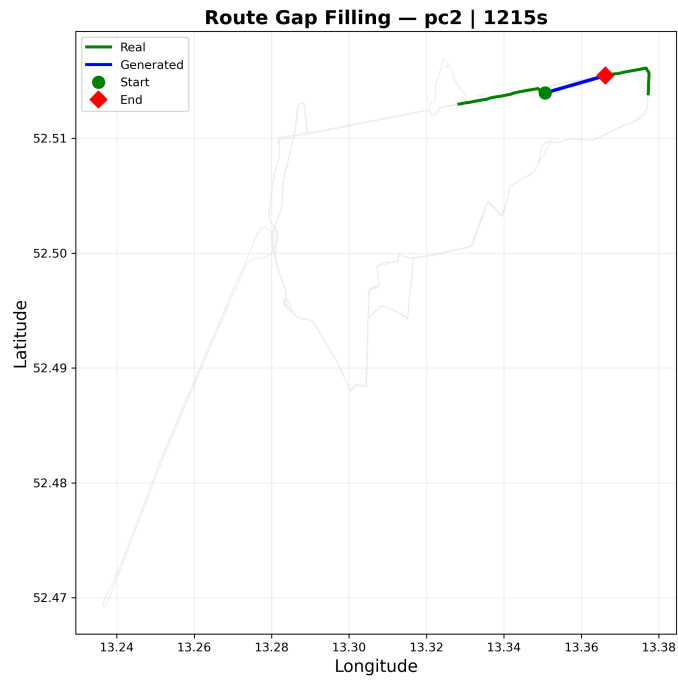


Fig. 5.8: Generated route for (a) Device 1 (b) Device 2

6 Conclusion

Vehicular communication measurement campaigns are expensive to conduct, limited to predefined routes, and inherently prone to data gaps. During real-world measurements, devices may lose connectivity, recording sessions may be interrupted, and even well-designed campaigns such as the Berlin V2X dataset can contain several minutes of missing data within a single measurement day. This thesis addressed these challenges through a two-part machine learning pipeline: first, by recovering missing measurements using data imputation, and second, by generating statistically realistic synthetic data to fill missing timestamp intervals.

The first part of the thesis demonstrated that missing cellular measurements can be effectively reconstructed using GRU-D. Unlike simpler imputation methods that mainly rely on feature-level correlations, GRU-D captures the temporal structure of measurement data and uses missingness patterns as informative signals. This makes the model particularly effective for high-priority network parameters such as RSRP, RSRQ, RSSI, and SNR. These signal-quality features are strongly influenced by previous measurements and thus benefit from a sequential model. In contrast, row-independent models, such as kNN and DAE, cannot explicitly utilise the temporal context prior to a missing gap.

The second part of the thesis showed that the proposed GRU-GAN model can address synthetic data generation and timestamp gap filling. The model captures the temporal behaviour and natural fluctuations of key features, including signal quality indicators, radio resource scheduling metrics, and QoS parameters. The recurrent generator is combined with adversarial training, ensuring that the generated sequences are close to the expected values and statistically realistic. This was also supported by the discriminative evaluation, in which the final discriminative score was reduced to 0.093, indicating that the generated sequences were close to real measurements. This result was also confirmed by the practical gap-filling experiment on the Berlin V2X dataset on selected signal and QoS features.

However, some limitations remain. Features with rapid directional changes, such as COG, and frequency parameters such as *PCell_freq_MHz*, *PCell_Downlink_frequency* are more difficult to reproduce accurately using sequential context alone. In addition, the current GRU-GAN generates sequences conditioned on a seed window selected from the same dataset and measurement route. A possible future extension would be to generate data for new timestamps or unseen routes, where the model must extrapolate beyond the distribution observed during training. According to Lu et al., outputs outside the training distribution are not guaranteed to be accurate, making extrapolation

a continuing challenge for generative models[45]. Therefore, future work could explore using additional conditioning information, such as route metadata or map embeddings, to enable controlled generation in new measurement scenarios.

The current pipeline was developed and tested on the V2I cellular dataset, a subset of the Berlin V2X dataset. This work can be further extended to V2V communication datasets, where the behaviour of the signal might differ due to the absence of established infrastructure, thereby extending the applicability of the proposed approach. Recent work on GAN-based synthetic data generation further demonstrates the feasibility of this approach[70].

Longer sequence generation remains an open problem. The current model generates short sequences from a 20-step seed window, where errors do not compound significantly. Extending the model to create longer sequences would necessitate minimizing propagation of errors during autoregressive generation, a well-known constraint of recurrent generative models[68].

Finally, the current pipeline is a post-processing framework, where gaps in the data are identified and reconstructed after the measurement process is completed. For implementing the approach to real-time network monitoring, where missing data are detected and filled in in real time during the measurements, low-latency inference capabilities and continuous online updates of the model would be required. DeepAuto demonstrates the potential for applying deep learning algorithms to the real-time prediction of cellular KPIs, suggesting that the real-time deployment of such algorithms is a realistic direction for future research[75]. Thus, with these future research opportunities, synthetic data generation approaches have great potential to promote reliable reconstruction and real-time generation of vehicular communication data for future intelligent transportation systems.

Bibliography

- [1] R. Hussain and S. Zeadally, “Autonomous cars: Research results, issues, and future challenges,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1275–1313, 2018.
- [2] M. Sadaf, Z. Iqbal, A. R. Javed, I. Saba, M. Krichen, S. Majeed, and A. Raza, “Connected and automated vehicles: Infrastructure, applications, security, critical challenges, and future aspects,” *Technologies*, vol. 11, no. 5, p. 117, 2023.
- [3] D. Wang, A. Qiu, Q. Zhou, and H. D. Schotten, “A Survey on the Role of Artificial Intelligence and Machine Learning in 6G-V2X Applications,” in *2025 IEEE 11th World Forum on Internet of Things (WF-IoT)*, 2025, pp. 1–7.
- [4] F. Harry, “Radio warning system for use on vehicles,” Dec. 28 1926, uS Patent 1,612,427.
- [5] Z. MacHardy, A. Khan, K. Obana, and S. Iwashina, “V2X Access Technologies: Regulation, Research, and Remaining Challenges,” *IEEE Communications Surveys & Tutorials*, vol. 20, no. 3, pp. 1858–1877, 2018.
- [6] ISO/TC 204, “Intelligent Transport Systems,” [Online]. Available: <https://www.iso.org/committee/54706.html>, accessed: Feb. 27, 2017.
- [7] S. Sesia, I. Toufik, and M. Baker, *LTE-the UMTS long term evolution: from theory to practice*. John Wiley & Sons, 2011.
- [8] M. Christopoulou, S. Barmounakis, H. Koumaras, and A. Kaloxylou, “Artificial Intelligence and Machine Learning as key enablers for V2X communications: A comprehensive survey,” *Vehicular Communications*, vol. 39, p. 100569, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214209622001164>
- [9] X. Zhang, J. Li, J. Zhou, S. Zhang, J. Wang, Y. Yuan, J. Liu, and J. Li, “Vehicle-to-everything communication in intelligent connected vehicles: a survey and taxonomy,” *Automotive Innovation*, vol. 8, no. 1, pp. 13–45, 2025.
- [10] H. Qian, M. Wang, M. Zhu, and H. Wang, “A review of multi-sensor fusion in autonomous driving,” *Sensors*, vol. 25, no. 19, p. 6033, 2025.

- [11] S. Campbell, N. O'Mahony, L. Krpalcova, D. Riordan, J. Walsh, A. Murphy, and C. Ryan, "Sensor technology in autonomous vehicles: A review," in *2018 29th Irish Signals and Systems Conference (ISSC)*. IEEE, 2018, pp. 1–4.
- [12] C. Ilas, "Electronic sensing technologies for autonomous ground vehicles: A review," in *2013 8th International Symposium on Advanced Topics in Electrical Engineering (ATEE)*. IEEE, 2013, pp. 1–6.
- [13] J. Lee, Y.-A. Li, M.-H. Hung, and S.-J. Huang, "A fully-integrated 77-GHz FMCW radar transceiver in 65-nm CMOS technology," *IEEE Journal of Solid-State Circuits*, vol. 45, no. 12, pp. 2746–2756, 2010.
- [14] B. Shahian Jahromi, T. Tulabandhula, and S. Cetin, "Real-time hybrid multi-sensor fusion framework for perception in autonomous vehicles," *Sensors*, vol. 19, no. 20, p. 4357, 2019.
- [15] T. Petrov, P. Pocta, and T. Kovacikova, "Benchmarking 4G and 5G-based cellular-V2X for vehicle-to-infrastructure communication and urban scenarios in cooperative intelligent transportation systems," *Applied Sciences*, vol. 12, no. 19, p. 9677, 2022.
- [16] G. Kumar and S. Mikkili, "Critical Review of Vehicle-to-Everything (V2X) Topologies: Communication, Power Flow Characteristics, Challenges, and Opportunities," *CPSS Transactions on Power Electronics and Applications*, vol. 9, no. 1, pp. 10–26, 2024.
- [17] O. Elma, U. Cali, and M. Kuzlu, "An overview of bidirectional electric vehicles charging system as a Vehicle to Anything (V2X) under Cyber-Physical Power System (CPPS)," *Energy Reports*, vol. 8, pp. 25–32, 2022, 2022 7th International Conference on Green Energy Technologies. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352484722019394>
- [18] D. Hur, D. Lee, J. Oh, D. Won, C. Song, and S. Cho, "Survey on challenges and solutions of C-V2X: LTE-V2X communication technology," in *2023 Fourteenth International Conference on Ubiquitous and Future Networks (ICUFN)*. IEEE, 2023, pp. 639–641.
- [19] K. Abboud, H. A. Omar, and W. Zhuang, "Interworking of DSRC and cellular network technologies for V2X communications: A survey," *IEEE transactions on vehicular technology*, vol. 65, no. 12, pp. 9457–9470, 2016.
- [20] F. Arena, G. Pau, and A. Severino, "A review on IEEE 802.11p for intelligent transportation systems," *Journal of Sensor and Actuator Networks*, vol. 9, no. 2, p. 22, 2020.

- [21] I. Series, “Mobile, radio determination, amateur and related satellite services,” *Detailed specifications of the terrestrial radio interfaces of IMT-2020*, Recommendation ITU, pp. 2150–1, 2015.
- [22] A. Martinez, E. Canibano, and J. Romo, “Analysis of low cost communication technologies for V2I applications,” *Applied Sciences*, vol. 10, no. 4, p. 1249, 2020.
- [23] A. Srivastav and S. Mandal, “Radars for Autonomous Driving: A Review of Deep Learning Methods and Challenges,” *IEEE Access*, vol. 11, pp. 97 147–97 168, 2023.
- [24] B. S. Bari, D. Puthal, and K. Yelamarthi, “Datasets in Vehicular Communication Systems: A Review of Current Trends and Future Prospects,” *SN Computer Science*, vol. 6, no. 3, p. 210, 2025.
- [25] N. Algeri and A. Boukerche, “A two-tier machine learning-based handover management scheme for intelligent vehicular networks,” *Ad Hoc Networks*, vol. 94, p. 101930, 2019.
- [26] J. Gao, M. R. A. Khandaker, F. Tariq, K.-K. Wong, and R. T. Khan, “Deep Neural Network Based Resource Allocation for V2X Communications,” in *2019 IEEE 90th Vehicular Technology Conference (VTC2019-Fall)*, 2019, pp. 1–5.
- [27] C. Wu, T. Yoshinaga, Y. Ji, and Y. Zhang, “Computational Intelligence Inspired Data Delivery for Vehicle-to-Roadside Communications,” *IEEE Transactions on Vehicular Technology*, vol. 67, no. 12, pp. 12 038–12 048, 2018.
- [28] L. Torres-Figueroa, H. F. Schepker, and J. Jiru, “QoS Evaluation and Prediction for C-V2X Communication in Commercially-Deployed LTE and Mobile Edge Networks,” in *2020 IEEE 91st Vehicular Technology Conference (VTC2020-Spring)*, 2020, pp. 1–7.
- [29] scikit-learn developers, “1.6. Nearest Neighbors — scikit-learn 1.8.0 documentation,” <https://scikit-learn.org/stable/modules/neighbors.html>, 2024, accessed: 2024-05-14.
- [30] —, “1.4. Support Vector Machines — scikit-learn 1.8.0 documentation,” <https://scikitlearn.org/stable/modules/svm.html>, 2026, accessed: 2026-05-14.
- [31] —, “1.17. Neural network models (supervised) — scikit-learn 1.8.0 documentation,” https://scikitlearn.org/stable/modules/neural_networks_supervised.html, 2026, accessed: 2026-05-14.

- [32] J. Schmidhuber, “Deep learning in neural networks: An overview,” *Neural networks*, vol. 61, pp. 85–117, 2015.
- [33] scikit-learn developers, “sklearn.cluster.KMeans — scikit-learn 1.8.0 documentation,” <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>, 2026, accessed: 2026-05-14.
- [34] scikit-learn-extra developers, “2. Clustering with KMedoids, CLARA and Common-nearest-neighbors — scikit-learn-extra 0.3.0 documentation,” <https://scikit-learn-extra.readthedocs.io/en/stable/modules/cluster.html>, 2019, accessed: 2026-05-14.
- [35] scikit-learn developers, “2.3. Clustering — scikit-learn 1.8.0 documentation,” <https://scikit-learn.org/stable/modules/clustering.html>, 2026, accessed: 2026-05-14.
- [36] Y. Zhang, X. Kong, W. Zhou, J. Liu, Y. Fu, and G. Shen, “A Comprehensive Survey on Traffic Missing Data Imputation,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 12, pp. 19 252–19 275, 2024.
- [37] J. C. Cole, “How to deal with missing data,” *Best practices in quantitative methods*, pp. 214–238, 2008.
- [38] S. Tak, S. Woo, and H. Yeo, “Data-driven imputation method for traffic data in sectional units of road links,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 6, pp. 1762–1771, 2016.
- [39] Q. Ma, W.-C. Lee, T.-Y. Fu, Y. Gu, and G. Yu, “MIDIA: exploring denoising autoencoders for missing data imputation,” *Data Mining and Knowledge Discovery*, vol. 34, no. 6, 2020.
- [40] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in *Proceedings of the 25th international conference on Machine learning*, 2008, pp. 1096–1103.
- [41] L. R. Medsker, L. Jain *et al.*, “Recurrent neural networks,” *Design and applications*, vol. 5, no. 64-67, p. 2, 2001.
- [42] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, “Recurrent neural networks for multivariate time series with missing values,” *Scientific reports*, vol. 8, no. 1, p. 6085, 2018.
- [43] A. R. Abdellah, A. Muthanna, M. H. Essai, and A. Koucheryavy, “Deep learning for predicting traffic in V2X networks,” *Applied Sciences*, vol. 12, no. 19, p. 10030, 2022.

- [44] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” *Advances in neural information processing systems*, vol. 27, 2014.
- [45] Y. Lu, L. Chen, Y. Zhang, M. Shen, H. Wang, X. Wang, C. van Rechem, T. Fu, and W. Wei, “Machine learning for synthetic data generation: a review,” *arXiv preprint arXiv:2302.04062*, 2023.
- [46] R. Xu, H. Xiang, Z. Tu, X. Xia, M.-H. Yang, and J. Ma, “V2X-vit: Vehicle-to-everything cooperative perception with vision transformer,” in *European conference on computer vision*. Springer, 2022, pp. 107–124.
- [47] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: An open urban driving simulator,” in *Conference on robot learning*. PMLR, 2017, pp. 1–16.
- [48] R. Xu, Y. Guo, X. Han, X. Xia, H. Xiang, and J. Ma, “OpenCDA: an open cooperative driving automation framework integrated with co-simulation,” in *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*. IEEE, 2021, pp. 1155–1162.
- [49] Y. Li, D. Ma, Z. An, Z. Wang, Y. Zhong, S. Chen, and C. Feng, “V2X-Sim: Multi-agent collaborative perception dataset and benchmark for autonomous driving,” *IEEE robotics and automation letters*, vol. 7, no. 4, pp. 10 914–10 921, 2022.
- [50] W. Zimmer, G. A. Wardana, S. Sritharan, X. Zhou, R. Song, and A. C. Knoll, “Tumtraf V2X cooperative perception dataset,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 22 668–22 677.
- [51] M. Skocaj, N. Di Cicco, T. Zugno, M. Boban, J. Blumenstein, A. Prokes, T. Mikulasek, J. Vychodil, K. Mikhaylov, M. Tornatore *et al.*, “Vehicle-to-everything (V2X) datasets for machine learning-based predictive quality of service,” *IEEE Communications Magazine*, vol. 61, no. 9, pp. 106–112, 2023.
- [52] Z. Pethő, T. M. Kazár, D. Vetési, T. Ormándi, Á. Török, and T. Tettamanti, “V2X measurement on M0 motorway with 4 vehicles,” 2023.
- [53] X. Zhang, L. Xiong, P. Zhang, H. Feng, J. Huang, X. Wang, M. Tian, Z. Li, and D. Yin, “5G communication delay dataset for cloud-based vehicle planning and control,” *Scientific Data*, 2026.

- [54] R. Hernangómez, P. Geuer, A. Palaios, D. Schäufele, C. Watermann, K. Taleb-Bouhemadi, M. Parvini, A. Krause, S. Partani, C. Vielhaus, M. Kasparick, D. F. Külzer, F. Burmeister, F. H. P. Fitzek, H. D. Schotten, G. Fettweis, and S. Stańczak, “Berlin V2X: A Machine Learning Dataset from Multiple Vehicles and Radio Access Technologies,” in *2023 IEEE 97th Vehicular Technology Conference (VTC2023-Spring)*, 2023, pp. 1–5.
- [55] M. Saravanan, V. Rajagopalan, and D. Sachdeva, “Understanding network nodal points for emergency services,” in *2023 15th international conference on communication systems & networks (COMSNETS)*. IEEE, 2023, pp. 847–851.
- [56] M. Saravanan and V. Pathmanaban, “Improving QoS in V2X Communication by observing Area-Level Network Nodal Points,” in *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*. IEEE, 2024, pp. 1–6.
- [57] N. Gotseva, A. Vlahov, V. Poulkov, and A. Manolova, “ML-driven Prediction of QoS in C-V2X Scenarios,” in *2024 59th International Scientific Conference on Information, Communication and Energy Systems and Technologies (ICEST)*. IEEE, 2024, pp. 1–4.
- [58] S. Partani, M. Zentarra, A. Kiggundu, and H. D. Schotten, “Improving QoS prediction in urban V2X networks by leveraging data from leading vehicles and historical trends,” in *2025 IEEE 101st Vehicular Technology Conference (VTC2025-Spring)*. IEEE, 2025, pp. 1–6.
- [59] B. Denizer and O. Landsiedel, “BandSeer: Bandwidth Prediction for Cellular Networks,” in *2024 IEEE 49th Conference on Local Computer Networks (LCN)*, 2024, pp. 1–8.
- [60] A. Graves, S. Fernández, and J. Schmidhuber, “Bidirectional LSTM networks for improved phoneme classification and recognition,” in *International conference on artificial neural networks*. Springer, 2005, pp. 799–804.
- [61] M. Patel, T. Patel, K. Patel, A. Nair, T. Vyas, S. Desai, and S. Tanwar, “BiLQoS: Stacked Bi-LSTM for Operator-Agnostic QoS Prediction in C-V2X Smart Cities,” in *2025 IEEE 4th Industrial Electronics Society Annual On-Line Conference (ONCON)*, 2025, pp. 1–6.
- [62] MobileInsight Team, “MobileInsight: Extracting and Analyzing Cellular Network Information on Smartphones,” <http://www.mobileinsight.net/>, 2022, accessed: 2026-05-14.
- [63] The Tcpdump Group, “Home | TCPDUMP & LIBPCAP,” <https://www.tcpdump.org/>, 2026, accessed: 2026-05-14.

- [64] iPerf Team, “iPerf - iPerf3 and iPerf2 user documentation,” <https://iperf.fr/iperf-doc.php>, 2026, accessed: 2026-05-14.
- [65] R. Hernangomez, P. Geuer, A. Palaos, D. Schäufole, C. Watermann, K. Taleb-Bouhemadi, M. Parvini, A. Krause, S. Partani, C. Vielhaus, M. Kasparick, D. F. Külzer, F. Burmeister, F. H. P. Fitzek, H. D. Schotten, G. Fettweis, and S. Stanczak, “Berlin V2X,” 2022. [Online]. Available: <https://dx.doi.org/10.21227/8cj7-q373>
- [66] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, “Recurrent neural networks for multivariate time series with missing values,” *Scientific reports*, vol. 8, no. 1, p. 6085, 2018.
- [67] K. Cho, B. Van Merriënboer, Ç. Gulçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1724–1734.
- [68] A. Graves, “Generating sequences with recurrent neural networks,” *arXiv preprint arXiv:1308.0850*, 2013.
- [69] J. Yoon, D. Jarrett, and M. van der Schaar, “Time-series Generative Adversarial Networks,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: <https://papers.nips.cc/paper/8789>
- [70] D. Tamayo-Urgilés, S. Sanchez-Gordon, Á. L. Valdivieso Caraguay, and M. Hernández-Álvarez, “GAN-based generation of synthetic data for vehicle driving events,” *Applied Sciences*, vol. 14, no. 20, p. 9269, 2024.
- [71] Y. Cheng *et al.*, “CausalTime: Realistically Generated Time-Series for Benchmarking of Causal Discovery,” in *International Conference on Learning Representations*, 2024.
- [72] A. Figueira and B. Vaz, “Survey on synthetic data generation, evaluation methods and GANs,” *Mathematics*, vol. 10, no. 15, p. 2733, 2022.
- [73] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://www.researchgate.net/publication/13853244_Long_Short-Term_Memory
- [74] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *International Conference on Learning Representations (ICLR)*, 2014. [Online]. Available: <https://arxiv.org/abs/1312.6114>

- [75] A. Bhorkar, K. Zhang, and J. Wang, “DeepAuto: A hierarchical deep learning framework for real-time prediction in cellular networks,” *arXiv preprint arXiv:2001.01553*, 2019.